Plataforma distribuida de interoperabilidad clínica para una red de IPS con tres sedes heterogéneas

Autores: Canchila, Jose Guillermo; Manjarrez, Sharit Javier

Sincelejo, Colombia

Correo electrónico: [correo institucional]

Asignatura: Sistemas Distribuidos · Parcial final · Noviembre de 2026

Versión: prototipo v3 (57 campos, traductor universal, sedes con motores distintos y seguridad OAuth 2.0 + TLS)

CONTENIDO

RESUMEN / ABSTRACT 4

INTRODUCCIÓN 5

OBJETIVOS 5

MARCO TEÓRICO 6

MARCO CONCEPTUAL 8

ESTADO DEL ARTE 9

SOLUCIÓN PROPUESTA Y METODOLOGÍA 10

- Arquitectura final 10
- Tecnologías empleadas 12
- Capa de datos 12
- Traductor universal de 57 campos 12
- Bus de servicios de salud (HSB) 13
- Seguridad, cuentas de usuario y panel de administración 13
- Seguridad v3: OAuth 2.0, bases de datos y cifrado entre servidores 14
- Metodología y hoja de ruta 16

MODELADO UML 16

- Diagrama de Casos de Uso 16
- Diagrama de Secuencias 18
- Diagrama de Clases 21
- Diagrama de Estados 22

PROCESO DE DESARROLLO 23
DISEÑO DE INTERFACES E IMPLEMENTACIÓN 24
Inicio de sesión y diseño adaptable 25
Panel de administración 26
Admisión y cuenta del paciente 30
Portal del paciente 33
Consulta médica 34
Reporte de historia clínica y estado de la plataforma 35
Experiencia de usuario y accesibilidad 37
PROTOTIPO FINAL 38
RESULTADOS Y DISCUSIÓN 42
Pruebas funcionales en vivo 42
Pruebas de seguridad 43
Indicadores operativos del bus 45
Discusión 46
CONCLUSIONES 47
Limitaciones 47
Trabajo futuro 48
REFERENCIAS 48
ANEXOS 49
Anexo A. Código clave 49
Anexo B. Pruebas automatizadas 49
Anexo C. Evidencias de ejecución 50
Anexo D. Usuarios de prueba 53
Anexo E. Cómo ejecutar 54
Anexo F. Uso declarado de inteligencia artificial 54
AVANCE INICIAL 55
VIABILIDAD TÉCNICA: ESCALABILIDAD Y CUMPLIMIENTO NORMATIVO 55
Escalabilidad 56
Cumplimiento normativo 56

INTEROPERABILIDAD BASADA EN DATOS: ENFOQUE SEMÁNTICO, TÉCNICO Y ORGANIZACIONAL 57
1. Calidad y Gestión de Datos 57
2. Interoperabilidad Semántica 57
3. Interoperabilidad Técnica 58
4. Gobernanza Organizacional 58
5. Herramientas de Integración Técnica 59
6. Herramientas de Integración Semántica 59
7. Compatibilidad de Herramientas con Estándares de Interoperabilidad 59
VALIDACIÓN DE MODELOS UML 60
Métodos 60
Resultados 61
Discusión 61

RESUMEN / ABSTRACT

Una red de IPS con sedes en Sincelejo, Cartagena y Corozal guarda la historia clínica de cada paciente fraccionada en sistemas legados heterogéneos. El objetivo fue construir una plataforma distribuida que unifique esa historia sin reemplazar los sistemas legados. Se implementó un bus de servicios de salud (HSB) en FastAPI que recibe HL7 v2, CSV y JSON, los traduce con un catálogo de los 57 campos de la Resolución 866 de 2021 a un registro canónico, los valida y los entrega como Bundle FHIR R4 a HAPI FHIR y a la base de la sede: PostgreSQL en Sincelejo y Corozal, MongoDB en Cartagena. Una API con OAuth 2.0, roles y un panel de administración consulta las sedes en paralelo y registra cada acceso en una cadena de auditoría SHA-256; el tráfico va cifrado con TLS y cada servicio usa un rol de mínimo privilegio. En una máquina virtual Kali Linux, el mensaje HL7 de prueba cubrió 56 de 57 campos y el 100 % de los obligatorios; la historia del paciente 1001 reunió 52 de 57 campos desde dos motores y su Bundle de 26 recursos validó en HAPI sin errores. Reenviar un mensaje no creó duplicados; con HAPI apagado, el mensaje se entregó solo tras 7 intentos (unos 130 s); al detener la sede MongoDB, la consulta respondió con dos sedes y marcó el reporte incompleto; el verificador señaló el evento alterado. Pasaron 60 pruebas automatizadas. Un modelo canónico y entregas idempotentes integran motores heterogéneos sin transacciones distribuidas.

Palabras clave: sistemas distribuidos, interoperabilidad clínica, HL7 v2, FHIR R4, Resolución 866 de 2021, MongoDB, teorema CAP, idempotencia, OAuth 2.0.

Abstract

A network of healthcare providers with sites in Sincelejo, Cartagena and Corozal (Colombia) keeps each patient's clinical record split across legacy systems with different databases and message formats. The goal was to build a distributed platform that unifies that record without replacing the legacy systems. A health service bus (HSB) written in FastAPI receives HL7 v2, CSV and JSON, translates them through a single catalogue of the 57 fields of Colombian Resolution 866 of 2021 into a canonical record, validates them and delivers them as FHIR R4 transaction Bundles to HAPI FHIR and to each site's database (PostgreSQL in two sites, MongoDB in the third). An API with OAuth 2.0 (RS256 tokens), four clinical roles and an administration panel serves the role-based views, queries the three sites in parallel and logs every access in a SHA-256 hash chain; traffic between servers is encrypted with TLS and each service uses its own least-privilege database role. On a Kali Linux virtual machine with 3.8 GB of RAM, the full test HL7 message covered 56 of 57 fields; patient 1001 gathered 52 of 57 fields from two database engines and his 26-resource Bundle validated in HAPI with no errors. Resending a message created no duplicates; with HAPI down, the message was delivered automatically after 7 attempts (about 130 s); with the MongoDB site stopped, the query answered with two sites and flagged the report as incomplete; the verifier pointed to the altered audit event. All 60 automated tests and 70 of 70 per-site checks passed.

Keywords: distributed systems, clinical interoperability, HL7 v2, FHIR R4, Resolution 866 of 2021, MongoDB, CAP theorem, idempotency, OAuth 2.0.

INTRODUCCIÓN

Cuando un paciente se atiende en Sincelejo, consulta por urgencias en Cartagena y hace sus controles en Corozal, cada sede guarda su parte de la historia clínica en su propio sistema. Ninguna sede ve la historia completa: se repiten exámenes, se pierde el registro de alergias o medicamentos y se toman decisiones con información parcial. Este es el problema de la **historia clínica fraccionada**.

El problema es relevante en Colombia porque la Resolución 866 de 2021 del Ministerio de Salud y Protección Social define el conjunto de 57 elementos de datos clínicos que deben poder intercambiarse entre prestadores (Ministerio de Salud y Protección Social, 2021). Cumplirla exige que sistemas construidos en épocas distintas, con bases y formatos distintos, hablen un mismo idioma sin que cada IPS tenga que reemplazar su sistema.

El reto del parcial es unificar la historia **sin reemplazar** los sistemas legados: cada sede conserva su base de datos, su motor y su formato de envío (HL7 v2 o CSV). En la versión 2 del proyecto se tomó esa heterogeneidad de forma literal, por indicación del docente: las sedes no usan el mismo motor. Sincelejo y Corozal usan PostgreSQL; Cartagena usa MongoDB, con un documento por paciente y nombres de campo distintos. La plataforma no puede suponer un esquema común.

La motivación técnica es aplicar en un sistema real los conceptos del curso de Sistemas Distribuidos: fallos parciales, el compromiso entre consistencia y disponibilidad (teorema CAP), transacciones ACID locales, fragmentación de datos por sede, mensajería con reintentos e idempotencia, y trazabilidad verificable. El resultado es un prototipo que se levanta con un solo comando, integra tres sedes con dos motores de base de datos, publica la información en FHIR R4 y ofrece vistas por rol (paciente, admisionista, médico, reporte de historia clínica) y un panel de administración.

El informe sigue la plantilla v7: primero el marco teórico, el marco conceptual y el estado del arte; luego la solución, el modelado UML, el proceso de desarrollo, las interfaces y el prototipo; después los resultados medidos, las conclusiones y los anexos, y al final las secciones de avance inicial, viabilidad técnica, interoperabilidad basada en datos y validación de los modelos UML.

OBJETIVOS

Objetivo general

Diseñar, implementar y evaluar una plataforma distribuida que unifique la historia clínica de un paciente atendido en tres sedes con sistemas legados heterogéneos (HL7 v2 y CSV;

PostgreSQL y MongoDB), usando FHIR R4 y los 57 campos de la Resolución 866 de 2021, sin reemplazar los sistemas legados.

Objetivos específicos

1. **Traducir** mensajes HL7 v2 (ADT^A01, ADT^A03, ORU^R01), filas CSV legadas y registros JSON de las tres sedes a un registro canónico de 57 campos, y medir la cobertura de campos por paciente (meta: 100 % de los campos obligatorios en el mensaje de prueba completo).
2. **Publicar** cada atención en HAPI FHIR R4 como Bundle `transaction` con `PUT` e identificadores deterministas, con 0 errores en la operación `$validate` y 0 recursos duplicados al reenviar un mensaje.
3. **Reducir a una sola consulta** la búsqueda de la historia de un paciente con registros en tres sedes y dos motores, de modo que siga respondiendo si una sede cae y avise que el reporte está incompleto.
4. **Controlar el acceso y la trazabilidad**: roles con permisos validados en la API (acceso denegado con código 403 y auditado), cuentas de usuario gestionadas por sede, auditoría encadenada por hash que señale el evento alterado exacto, y entrega automática de un mensaje cuando HAPI vuelve tras una caída.

MARCO TEÓRICO

Esta sección resume los fundamentos que sustentan el diseño. Las definiciones operativas de los términos que se usan en el proyecto están en el marco conceptual.

Sistemas distribuidos, transparencia y fallos parciales

Un sistema distribuido es un conjunto de computadores autónomos que el usuario percibe como un solo sistema coherente (Tanenbaum y Van Steen, 2017). La transparencia de ubicación oculta dónde están los datos; la transparencia de fallos busca que el sistema siga operando aunque falle una parte. Lo distintivo de un sistema distribuido es el **fallo parcial**: una sede puede caer mientras las otras siguen funcionando. En este proyecto el fallo no se oculta: la plataforma responde con lo que tiene y dice qué falta.

Teorema CAP

Ante una partición de red, un sistema distribuido debe elegir entre consistencia (todas las réplicas ven el mismo dato) y disponibilidad (toda petición recibe respuesta). La consulta unificada de la plataforma es **AP**: si la sede 2 no responde, la historia se arma con las sedes 1 y 3 y se marca `reporte_completo = false`. Lo que se sacrifica es ver, en ese momento, las atenciones de la sede caída. La escritura es **eventualmente consistente**: el bus escribe primero en HAPI y luego en la sede; si la sede falla, el mensaje queda pendiente y se completa cuando la sede vuelve.

Transacciones ACID

Atomicidad, consistencia, aislamiento y durabilidad describen una transacción local. En las sedes PostgreSQL, el usuario, la atención, los diagnósticos, las tecnologías en salud y el egreso se guardan en una sola transacción: o se guarda todo o no se guarda nada. En MongoDB la unidad atómica es el documento; como la atención completa está embebida en el documento del paciente, guardarla es una sola escritura atómica, sin transacciones multi-documento.

Fragmentación horizontal y consulta distribuida

La fragmentación horizontal parte los datos por filas según un criterio. Aquí el criterio es la sede de la atención: cada sede guarda sus propias atenciones y la clave común entre sedes es el número de documento del paciente. Para reconstruir la historia, la API consulta las tres sedes al mismo tiempo y une los resultados en memoria (patrón *scatter-gather*; Hohpe y Woolf, 2003).

Persistencia políglota

Usar motores de base de datos distintos según la necesidad de cada parte del sistema. Un modelo relacional normaliza la atención en varias tablas con llaves; un modelo documental guarda al paciente y sus atenciones en un solo documento. La plataforma no cambia los motores de las sedes: los trata a través de una interfaz común.

HL7 versión 2

Estándar de mensajería clínica por segmentos de texto (`MSH`, `PID`, `PV1`, `DG1`, `OBX`, `RXE`…), con campos separados por `|` y componentes por `^`. El campo `MSH-10` identifica cada mensaje. Los datos que el estándar no trae se envían en **segmentos Z**, definidos por acuerdo entre las partes.

HL7 FHIR R4

Estándar de HL7 basado en recursos (Patient, Encounter, Condition, Observation, MedicationRequest…) que se intercambian por una API REST en JSON (HL7 International, 2019). Un Bundle de tipo `transaction` agrupa recursos que el servidor procesa como una unidad. Frente a la sintaxis de HL7 v2, FHIR ofrece recursos con identidad propia y operaciones web estándar (Bender y Sartipi, 2013).

Bus de integración e idempotencia

Un bus de integración desacopla a quien envía de quien recibe y concentra la traducción, la validación, el enrutamiento y los reintentos (Hohpe y Woolf, 2003). Cuatro patrones de esa obra guían el diseño: **modelo canónico de datos** (un formato pivote entre todos los formatos), **receptor idempotente** (un mensaje repetido no cambia el resultado), **canal de mensajes muertos** (los mensajes que no se pueden entregar se apartan para revisión) y **scatter-gather**. Una operación es idempotente si repetirla deja el mismo estado que hacerla una vez; es lo que permite reintentar sin miedo a duplicar.

Integridad por encadenamiento de hash

Si cada registro de auditoría incluye el hash del registro anterior, modificar, borrar o reordenar un registro rompe la cadena desde ese punto. Recalcular la cadena permite señalar el primer registro alterado. Es el mismo principio de los registros encadenados, aplicado aquí sobre una tabla de PostgreSQL y no sobre una red de *blockchain*.

MARCO CONCEPTUAL

La tabla siguiente define los conceptos clave tal como se usan en el proyecto e indica dónde aparece cada uno. Después se explica cómo se relacionan.

Concepto	Definición en el proyecto	Dónde aparece
FHIR R4	Estándar de HL7 para intercambiar información clínica como recursos por REST.	Servidor HAPI FHIR R4 v7.2.0 (`:8090/fhir`)
Recurso	Unidad de información FHIR con identidad propia: Patient, Encounter, Condition, Observation, MedicationRequest, AllergyIntolerance, Practitioner, Organization.	`fhir/normalizer.py`
Perfil y extensión	Un perfil restringe un recurso para un uso concreto; una extensión agrega datos que el recurso base no trae. El proyecto usa extensiones propias, no un perfil publicado.	URL base `https://minsalud.gov.co/fhir/StructureDefinition/`
Bundle `transaction`	Conjunto de recursos que HAPI procesa como una unidad; cada entrada usa `PUT` con un identificador determinista.	`pac-<doc>`, `enc-<visita>`, `cond-…`, `obs-…`
Terminología	Sistema de códigos con significado compartido: CIE-10 para diagnósticos (`I10`, `J45.9`) y LOINC para laboratorio (`718-7`).	Condition.code, Observation.code
Dominio (Res. 866)	Lista de valores permitidos de un campo (tipo de documento, entorno, triage, vía de ingreso…) con los sinónimos que usa cada sede.	`traductor/dominios.py`
API REST	Interfaz por HTTP con recursos y verbos estándar (GET, POST, PUT).	HSB (`:8001`), API Gateway (`:8000`), HAPI
HSB	Bus de servicios de salud: recibe, traduce, valida, normaliza, entrega y reintenta.	`hsb/` (FastAPI)
Catálogo de 57 campos	Fuente única de verdad: para cada campo de la Res. 866 indica su origen HL7, CSV, su destino PostgreSQL, MongoDB y FHIR, si es obligatorio y su dominio.	`traductor/catalogo_57.py`
Registro canónico	Representación intermedia y común de una atención con los 57 campos; pivote entre todos los formatos.	`traductor/traductor.py`
Traductor	Módulo que convierte HL7, CSV o JSON al registro canónico, y el canónico a filas PostgreSQL, documento MongoDB o Bundle FHIR.	`traductor/`
Segmento Z	Segmento HL7 v2 acordado entre las partes para datos sin lugar en el estándar.	`ZPI`, `ZPV`, `ZRX`, `ZEG`
Sede / nodo	IPS con su propia base de datos (fragmento horizontal de la historia).	`pg_sede1`, `mongo_sede2`, `pg_sede3`
Consulta unificada	Lectura en paralelo de las tres sedes que devuelve la historia y qué sedes respondieron.	`db/consulta_unificada.py`
Cobertura	De campos: cuántos de los 57 tienen valor para un paciente. FHIR por sede: pacientes con recurso en HAPI sobre pacientes de la sede.	Reporte de HC y tablero
Evento de auditoría y sello	Registro encadenado por SHA-256 de cada acceso o escritura; el hash del evento de una escritura es el sello que se devuelve al usuario.	`pg_central.evento_auditoria`

Concepto	Definición en el proyecto	Dónde aparece
Rol y cuenta de usuario	Paciente, admisionista, médico, auditor y administrador (general o de sede). Las cuentas se guardan en la base central con contraseña PBKDF2.	`pg_central.usuario_sistema`

Tabla 1. Conceptos clave del proyecto.

Relación entre los conceptos

Un sistema de sede envía un mensaje HL7 v2 o una fila CSV al **HSB** por su **API REST**. El **traductor** lo lleva al **registro canónico** usando el **catálogo de 57 campos**; los **segmentos Z** aportan lo que HL7 no trae y los **dominios** unifican los códigos locales. El bus valida las **terminologías** (CIE-10, LOINC), arma un **Bundle transaction** con **recursos** FHIR y sus **extensiones**, lo publica en HAPI y guarda el mismo registro en la base de su **sede**. Para leer, la **consulta unificada** reúne las tres sedes, calcula la **cobertura** y deja un **evento de auditoría**. Cada acción depende del **rol** de la cuenta que inició sesión.

ESTADO DEL ARTE

Se revisaron fuentes de alcance global, latinoamericano y colombiano sobre estándares y herramientas de interoperabilidad. La tabla resume qué aporta cada fuente al proyecto; las referencias completas están al final.

#	Fuente	Alcance	Qué se tomó para el proyecto
1	Bender y Sartipi (2013)	Global	Argumentos a favor de FHIR (recursos REST) frente a la sintaxis de HL7 v2: base para publicar en FHIR y conservar HL7 v2 solo como formato de entrada de los legados.
2	Centro Nacional en Sistemas de Información en Salud (2022)	Latinoamérica (Chile)	Uso de Observation con LOINC y Condition con CIE-10 en redes asistenciales.
3	HAPI FHIR Community (2024)	Global	Servidor FHIR R4 de referencia, operación `$validate` y Bundles `transaction`.
4	Hohpe y Woolf (2003)	Global	Patrones *Idempotent Receiver*, *Dead Letter Channel*, *Scatter-Gather* y *Canonical Data Model*, que el HSB implementa.
5	Ministerio de Salud de la Nación Argentina (2020)	Latinoamérica (Argentina)	Experiencia nacional de implementación de FHIR con identificadores estables.
6	Ministerio de Salud y Protección Social (2021)	Colombia	Resolución 866: los 57 datos de la historia clínica interoperable y sus dominios.
7	NextGen Healthcare (2023)	Global	Mirth Connect como motor de integración HL7 de referencia.
8	OpenHIE Consortium (2022)	Global / países en desarrollo	OpenHIM: separar la mediación (seguridad, auditoría) del almacenamiento.
9	HL7 International (2019)	Global	Especificación FHIR R4: recursos, Bundle, transacciones y extensiones.
10	Tanenbaum y Van Steen (2017)	Global	Fallos parciales, transparencia y consistencia en sistemas distribuidos.

Tabla 2. Fuentes del estado del arte y aporte al proyecto.

Análisis crítico

Las experiencias nacionales revisadas (Chile y Argentina) coinciden en publicar la información clínica en FHIR con terminologías estándar e identificadores estables, y en dejar HL7 v2 para los sistemas existentes. Bender y Sartipi (2013) explican por qué: FHIR expone recursos con identidad propia por REST, mientras que HL7 v2 depende de acuerdos de sintaxis entre cada par de sistemas. Las herramientas de integración (Mirth Connect, OpenHIM, Apache Camel) resuelven el transporte y la mediación, pero suponen una plataforma de ejecución adicional y una configuración propia.

La brecha que aborda este proyecto es más pequeña y concreta: integrar sedes que no comparten ni el formato ni el motor de base de datos, y medir el cumplimiento de los 57 campos de la Resolución 866 de 2021 de punta a punta, desde el segmento HL7 hasta el elemento FHIR. En las fuentes revisadas no se encontró un perfil FHIR colombiano publicado que el proyecto pudiera usar; por eso los campos sin lugar en el recurso base se envían como extensiones propias.

Comparación de herramientas

Criterio	HAPI FHIR	Mirth Connect	OpenHIM	Apache Camel	HSB propio (FastAPI)
Papel	Servidor FHIR	Motor de integración HL7	Mediador de intercambio	Marco de integración	Bus del proyecto
FHIR R4	Nativo	Por conectores	Hacia un servidor FHIR externo	Por componentes	Cliente REST hacia HAPI
HL7 v2	Librería de lectura	Nativo (canales)	No es su foco	Componente HL7	Traductor propio guiado por catálogo
Plataforma	Java	Java	Node.js	Java	Python
Configuración	Código / propiedades	Interfaz gráfica de canales	Consola web	DSL / código	Código y catálogo de 57 campos
Uso en el proyecto	**Sí** (servidor FHIR)	No	No	No	**Sí** (bus)

Tabla 3. Comparación de HAPI FHIR, Mirth Connect, OpenHIM, Apache Camel y el HSB del proyecto.

Se usa HAPI FHIR porque es la implementación abierta de referencia de FHIR R4, acepta Bundles `transaction`, ofrece `$validate` y el equipo ya la conocía del laboratorio. Para el bus se descartaron Mirth Connect, OpenHIM y Apache Camel porque cada integrante debe poder explicar y modificar en vivo cualquier parte del sistema (regla del parcial) y porque cada uno agrega otra plataforma de ejecución. A cambio, el proyecto renuncia a sus herramientas gráficas y a sus conectores ya construidos.

SOLUCIÓN PROPUESTA Y METODOLOGÍA

Arquitectura final

La plataforma son siete contenedores que se levantan con un solo comando (`docker compose down -v && docker compose up -d --build`). La Figura 1 muestra los componentes y los flujos; la tabla resume el papel de cada contenedor.

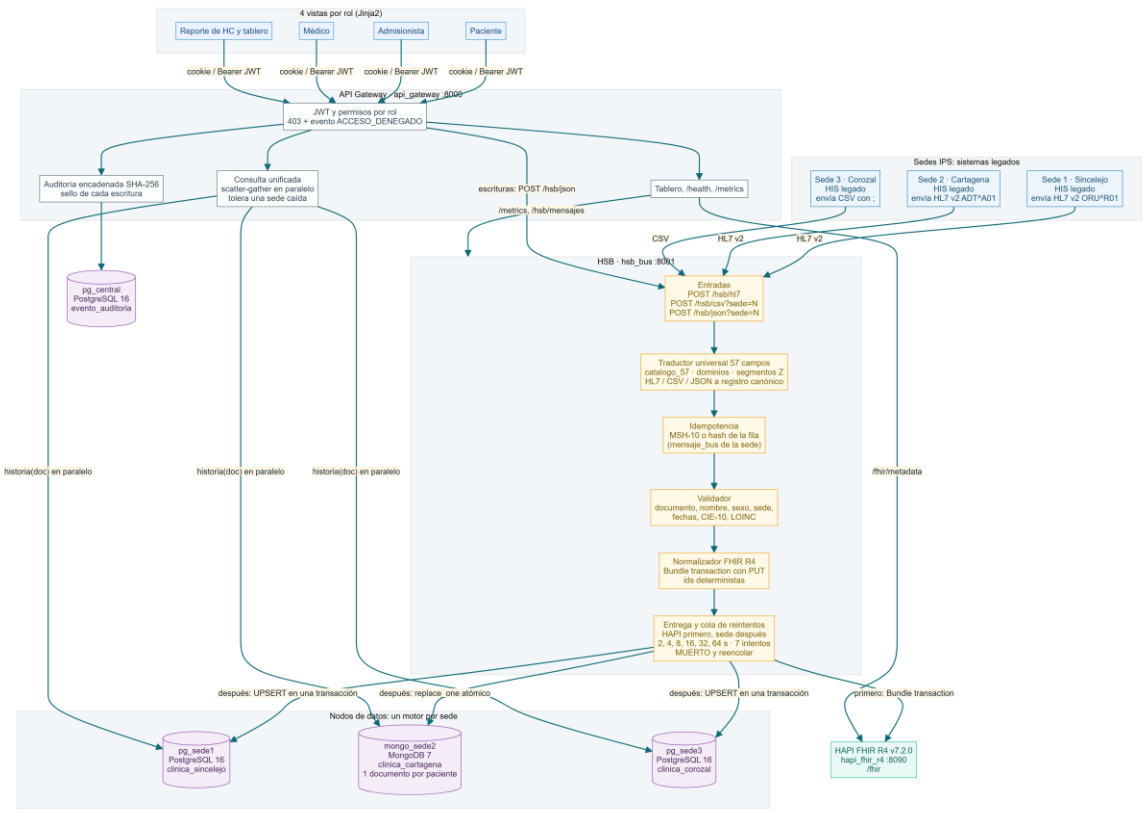


Figura 1. Arquitectura de la plataforma: tres sedes con su motor, HSB con traductor de 57 campos, HAPI FHIR R4, API Gateway con las vistas por rol y base central de auditoría y cuentas.

Contenedor	Papel	Puerto
`pg_sede1`	Sede 1 Sincelejo — PostgreSQL 16 (`clinica_sincelejo`)	Interno
`mongo_sede2`	Sede 2 Cartagena — MongoDB 7 (`clinica_cartagena`), un documento por paciente	Interno
`pg_sede3`	Sede 3 Corozal — PostgreSQL 16 (`clinica_corozal`)	Interno
`pg_central`	Base central: cadena de auditoría (`evento_auditoria`), cuentas de usuario (`usuario_sistema`), refresh tokens OAuth y base de HAPI. No guarda datos clínicos de las sedes.	Interno
`hapi_fhir_r4`	HAPI FHIR R4 v7.2.0, persistido en PostgreSQL	8090 (solo local)
`hsb_bus`	Health Service Bus (FastAPI); exige token OAuth 2.0	8001 HTTPS
`api_gateway`	API Gateway, servidor OAuth 2.0, vistas por rol y panel de administración (FastAPI + Jinja2)	8000 HTTPS

Tabla 4. Contenedores de la plataforma. Las bases viven en una red interna sin puertos publicados.

Regla de oro. Ninguna vista habla directo con las bases de las sedes. Las lecturas pasan por la consulta unificada de la API; las escrituras (admisión, diagnóstico, fórmula, egreso) viajan al HSB por `/hsb/json` y siguen el mismo camino que un mensaje HL7. El diagrama editable está en Mermaid en `docs/fase1/arquitectura/arquitectura_y_justificacion.md`, junto con la tabla que justifica cada componente frente al requisito que cubre.

Tecnologías empleadas

- **Lenguaje y servicios:** Python 3.11, FastAPI y Uvicorn para el HSB y la API; Jinja2 para las vistas.
- **Datos:** PostgreSQL 16 (sedes 1 y 3, base central), MongoDB 7 (sede 2), con psycopg2 y PyMongo.
- **Estándares:** HL7 v2.5 (ADT^A01, ADT^A03, ORU^R01, RDE^O11), FHIR R4 en HAPI FHIR 7.2.0, CIE-10 y LOINC.
- **Seguridad:** OAuth 2.0 con tokens JWT firmados con RS256 (python-jose y cryptography), HTTPS y TLS 1.3 con una autoridad certificadora propia (OpenSSL), SCRAM-SHA-256 en las bases, contraseñas PBKDF2-SHA256 (200 000 iteraciones, sal por usuario) y SHA-256 para la cadena de auditoría.
- **Despliegue y pruebas:** Docker Compose; manifiestos de Kubernetes para Minikube en `infra/k8s/`; `pytest`. Diagramas en Mermaid.

Capa de datos

- **Un nodo por sede (RF4).** Cada IPS conserva su base; no hay base central de datos clínicos.
- **Motores distintos.** Las sedes 1 y 3 usan el esquema relacional de 57 campos (`db/schema.sql`): `usuario`, `atencion`, `diagnostico`, `tecnologia_salud`, `egreso`, `profesional_salud`, `sede` y `mensaje_bus`. La sede 2 usa la colección `pacientes`, con un documento por paciente y sus atenciones embebidas, más `mensajes_bus`, `profesionales` y `sedes`. Los nombres no coinciden a propósito (`identificacion.numero` frente a `documento_id`).
- **Repositorio con una sola interfaz.** `RepositorioPostgres` y `RepositorioMongo` (`db/repositorio.py`) exponen los mismos métodos (`guardar`, `historia`, `cobertura_fhir` y los de `mensaje_bus`) y hablan siempre en el registro canónico. El bus y la API no saben qué motor tiene cada sede.
- **ACID local.** En PostgreSQL, `guardar` hace UPSERT de las cinco tablas en una transacción; en MongoDB reemplaza el documento completo del paciente con `replace_one`, que es atómico.
- **Sin transacción distribuida.** No hay 2PC entre sedes ni entre HAPI y la sede: la entrega es idempotente y se reintenta hasta completarse.

Traductor universal de 57 campos

- `catalogo_57.py` es la fuente única de verdad: para cada campo indica el segmento HL7, la columna CSV, la `tabla.columna` de PostgreSQL, la ruta del documento MongoDB y el elemento FHIR, si es obligatorio (23 lo son) y su dominio.
- `dominios.py` guarda los códigos permitidos y los sinónimos de cada sede (por ejemplo, `E` en PV1-2 equivale al entorno `01` Urgencias, el triage `III` a `3` y la vía `PO` a `01` Oral).

- `traductor.py` convierte HL7, CSV o JSON al registro canónico y este a filas PostgreSQL, documento MongoDB o Bundle FHIR. Con tres entradas y tres salidas se escriben seis traducciones y no nueve.
- Los campos que HL7 v2.5 no trae viajan en segmentos Z: `ZPI` (datos complementarios del paciente), `ZPV` (modalidad y triage), `ZRX` (finalidad del medicamento) y `ZEG` (egreso).
- Edad, unidad de edad y código del prestador se derivan; la cobertura de campos se calcula por paciente con la lista de faltantes. El mapeo completo se genera en `docs/mapeo_57_campos.md`.

Bus de servicios de salud (HSB)

Entradas `POST /hsb/hl7`, `POST /hsb/csv?sede=N` y `POST /hsb/json?sede=N`; operación `GET /hsb/mensajes`, `GET /hsb/catalogo`, `POST /hsb/reencolar/{id}`, `GET /health` y `GET /metrics`. El flujo de cada mensaje (`hsb/ingesta.py`, `hsb/queue_manager.py`) es:

1. **Traducir** al registro canónico.
2. **Verificar idempotencia:** busca `MSH-10` (o el hash SHA-256 de la fila CSV o del JSON) en `mensaje_bus` de la sede; si ya está ENTREGADO, TRANSFORMADO o REINTENTANDO responde `duplicado`.
3. **Validar** documento, nombre, sexo, sede, fechas, CIE-10 y LOINC; si algo falla, RECHAZADO con todos los motivos.
4. **Normalizar** a un Bundle `transaction` con `PUT` e identificadores deterministas.
5. **Entregar** primero a HAPI y después a la sede; ambos pasos son idempotentes.
6. **Reintentar** en segundo plano con esperas de 2, 4, 8, 16, 32 y 64 s (`HSB_MAX_RETRIES=7`); al agotarlos, MUERTO, con opción de reencolar. Los errores de datos (restricción violada, HTTP 4xx de HAPI) van directo a MUERTO. Al reiniciar, el bus recupera los mensajes pendientes de las sedes.

Seguridad, cuentas de usuario y panel de administración

- **Token OAuth 2.0** (JWT RS256) de 60 minutos emitido en `/auth/login` o en `/oauth/token`; viaja en cookie (`HttpOnly`, `Secure`) para las vistas o como `Bearer`. El detalle está en la subsección siguiente.
- **Roles validados en la API:** paciente (solo su historia), admisionista (búsqueda y admisión, sin diagnósticos), médico (historia unificada y escrituras clínicas), auditor (reporte, tablero y verificador; en el panel solo consulta) y administrador. Un acceso no permitido responde 403 y deja un evento `ACCESO_DENEGADO`.
- **Cuentas en la base central** (`usuario_sistema`) con contraseña PBKDF2-SHA256 y trazabilidad del registro: origen (`SEMILLA`, `PANEL` o `ADMISION`), quién la creó, cuándo y último acceso. El administrador general gestiona cuentas de toda la red; el administrador de sede solo crea y gestiona médicos, admisionistas y pacientes de su sede.

- **Cuenta del portal del paciente:** al admitir a un paciente nuevo se crea su cuenta (usuario = número de documento) con una contraseña temporal que se muestra una sola vez; en el primer ingreso el sistema obliga a cambiarla antes de permitir cualquier otra pantalla.
- **Auditoría encadenada** en `pg_central.evento_auditoria`: `hash_propio = SHA-256(hash_anterior | índice | fecha | usuario | rol | tipo | recurso | resultado | detalle)`, con `pg_advisory_xact_lock` para que dos inserciones simultáneas no enlacen al mismo hash anterior.
- **Panel de administración** (`/panel`): una pantalla que se actualiza cada 5 s con las secciones Resumen, Pacientes, Usuarios, Integración, FHIR y Auditoría.
- **Datos sintéticos:** `scripts/generar_datos_prueba.py` genera pacientes ficticios en las tres sedes y los envía por el bus (HL7 v2 con segmentos Z y CSV), incluidos errores y reenvíos a propósito; es determinista y no crea copias si se repite.

Seguridad v3: OAuth 2.0, bases de datos y cifrado entre servidores

A pedido del docente se agregaron tres capas de seguridad. Cada una es independiente de las otras: un atacante que pase una tiene que pasar también las demás.

Autenticación y autorización con OAuth 2.0

La API es el servidor de autorización (`app/oauth.py`) y el HSB es un servidor de recursos (`hsb/autorizacion.py`). Los tokens son JWT firmados con RS256: la API firma con su clave privada y el bus verifica con la clave pública que la API publica en `/oauth/jwks`, así que no comparten ningún secreto. Cada token lleva emisor (`iss`), audiencia (`aud`), vencimiento, identificador único (`jti`) y los `scope` que autoriza (Hardt, 2012).

Endpoint	Estándar	Para qué
`POST /oauth/token`	RFC 6749	Emite tokens con los flujos `password` (usuario), `client_credentials` (servicio a servicio) y `refresh_token` (renovación con rotación)
`GET /oauth/jwks`	RFC 7517	Clave pública RSA para verificar las firmas
`POST /oauth/introspect`	RFC 7662	Dice si un token está activo y qué contiene
`POST /oauth/revoke`	RFC 7009	Revoca un refresh token
`GET /.well-known/oauth-authorization-server`	RFC 8414	Metadatos del servidor

Tabla 5. Endpoints del servidor de autorización OAuth 2.0.

Cliente	Flujo	Scopes y audiencia	Uso
`portal_web`	password, refresh_token	Los del rol del usuario; `aud = api`	Inicio de sesión de las vistas web
`api_gateway`	client_credentials	`hsb.ingesta hsb.admin`; `aud = hsb`	La API escribe en el bus
`sistema_sede1` a `sistema_sede3`	client_credentials	`hsb.ingesta`; `aud = hsb`; `sede_id`	Sistema legado de cada sede: solo envía mensajes de su sede
`herramientas_qa`	password, client_credentials, refresh_token	`hsb.ingesta hsb.admin`	Postman, scripts de carga y de verificación

Tabla 6. Clientes OAuth 2.0 registrados.

- **El bus exige token.** `/hsb/hl7`, `/hsb/csv` y `/hsb/json` piden el scope `hsb.ingesta`; `/hsb/mensajes`, `/hsb/reencolar` y `/hsb/migrar` piden `hsb.admin`. Sin token responde 401 con `WWW-Authenticate: Bearer` (RFC 6750); con un scope insuficiente, 403 `insufficient_scope`. `/health`, `/metrics` y `/hsb/catalogo` siguen abiertos porque no tienen datos de pacientes.
- **Audiencia separada.** Un token de usuario (`aud = api`) no sirve en el bus, y uno de servicio (`aud = hsb`) no sirve en la API.
- **Restricción por sede.** El token de `sistema_sede1` lleva `sede_id = 1`; si intenta enviar un CSV de la sede 3, el bus responde 403.
- **Refresh tokens.** Son opacos y en `pg_central.token_refresco` se guarda solo su hash SHA-256; al usarse se revocan y se emite uno nuevo (rotación).
- **Pasarela FHIR.** HAPI FHIR no tiene autenticación propia, así que no se publica a la red: quien necesita FHIR desde fuera entra por `/fhir/...` de la API con un token de médico, auditor o administrador, y cada acceso queda en la auditoría.

Seguridad de las bases de datos

Antes, la API y el bus entraban a todas las bases con el superusuario. Ahora cada servicio tiene su propio rol con los permisos justos (`db/seguridad/`). Los roles se crean al iniciar una base vacía y `desplegar.sh` los vuelve a aplicar sobre bases existentes.

Base	Rol	Permisos	Lo que no puede hacer
Sedes PostgreSQL	`hsb_escritor` (bus)	SELECT, INSERT, UPDATE	DELETE, TRUNCATE, cambiar la estructura
Sedes PostgreSQL	`api_lector` (API)	SELECT	Escribir: las escrituras van por el bus
Sede 2 MongoDB	`hsb_escritor`	Rol `escritor_sin_borrado`: find, insert, update	remove, dropCollection
Sede 2 MongoDB	`api_lector`	Rol `solo_lectura`: find	insert, update, remove
Central	`api_central` (API)	Auditoría: SELECT e INSERT; cuentas y refresh tokens: SELECT, INSERT, UPDATE	Modificar o borrar la auditoría (solo agregar)
Central (base `hapi`)	`hapi_srv` (HAPI)	Dueño de su base	Entrar a `clinica_central`

Tabla 7. Roles de mínimo privilegio por base.

- **Sin exposición.** Las cuatro bases están en la red Docker interna `red_datos` (sin salida ni puertos en el anfitrión); HAPI solo escucha en `127.0.0.1`.
- **TLS obligatorio.** PostgreSQL rechaza cualquier conexión de red sin cifrado (`pg_hba.conf` con `hostssl` y `reject`) y autentica con SCRAM-SHA-256; MongoDB corre con `requireTLS`. El superusuario solo se usa desde dentro del contenedor, como DBA.
- **Auditoría de solo agregar.** Como la aplicación no puede hacer UPDATE ni DELETE sobre `evento_auditoria`, la única forma de alterarla es como DBA, y el verificador de la cadena lo detecta.
- **Secretos fuera del código.** Las contraseñas de los roles van en `.env` (no versionado); `desplegar.sh` las genera aleatorias al crearlo.

Cifrado de la información entre servidores

`scripts/generar_certificados.sh` crea una autoridad certificadora propia y un certificado por servidor, con los nombres de red de cada contenedor. Todos los clientes verifican el certificado del otro contra esa CA, así que un tercero no puede hacerse pasar por un servicio.

Tramo	Protocolo	Verificación
Navegador o Postman → API y HSB	HTTPS (TLS 1.3)	Certificado con las IP y nombres del servidor
API → HSB (escrituras, migración, monitor)	HTTPS + token `client_credentials`	CA propia + JWT RS256
HSB y API → HAPI FHIR	HTTPS (Spring Boot con PKCS#12)	CA propia
HSB y API → PostgreSQL de las sedes y central	TLS 1.3, SCRAM-SHA-256	`sslmode=verify-full`
HSB y API → MongoDB	TLS (`requireTLS`)	CA propia
HAPI → base `hapi`	TLS	`sslmode=require`

Tabla 8. Cifrado en tránsito por tramo de comunicación.

Metodología y hoja de ruta

Se trabajó de forma iterativa en cinco fases, cada una con un entregable funcional y revisión cruzada entre los dos integrantes. Cada fase partió del código de la anterior; las correcciones entre fases se anotaron en la bitácora (`docs/bitacora.md`). Después de la Fase 5 se hizo una mejora (v2) que reescribió la capa de datos, el bus y la auditoría.

Fase	Entregable	Estado
1. Análisis, UML y diseño	Informe, 4 diagramas, arquitectura, mapeo de campos	Entregada; diagramas actualizados a v2
2. Datos distribuidos y FHIR	Nodos por sede, normalizador FHIR, demostración ACID	Entregada; en v2, esquema de 57 campos y sede 2 en MongoDB
3. HSB	Bus HL7/CSV con reintentos e idempotencia	Entregada; en v2, cola en segundo plano, recuperación y `/hsb/json`
4. Seguridad y resiliencia	JWT, roles, auditoría, caída de sede	Entregada; en v2, auditoría persistida en `pg_central`
5. Prototipo final	Vistas, informe, sustentación	Este informe
Mejora v2 (10 y 11 de nov.)	Traductor de 57 campos, MongoDB, panel de administración, gestión de usuarios, generador de datos	Implementada y probada (44 pruebas)
Mejora v3 seguridad (11 de nov.)	OAuth 2.0, roles de mínimo privilegio en las bases, TLS entre servidores	Implementada y probada (60 pruebas, 70/70 comprobaciones)

Tabla 9. Hoja de ruta de las fases del proyecto.

MODELADO UML

Diagrama de Casos de Uso

El modelado se hizo con cuatro diagramas UML escritos en **Mermaid**. El código fuente editable está en `docs/fase1/diagramas/` (archivos `01_casos_uso.md`, `02_secuencia_hsb.md`,

`03_clases.md` y `04_estados_mensaje.md`) y las imágenes renderizadas en `docs/diagramas/`. Cada archivo fuente incluye una tabla de coherencia con los otros tres diagramas.

El diagrama de casos de uso (Figura 2) tiene cinco actores (paciente, admisionista, médico, auditor y sistema de sede) y 17 casos de uso. Los casos UC1 a UC13 corresponden a las vistas por rol; UC14 y UC15 al envío y procesamiento de mensajes por el bus; UC16 y UC17 los ejecuta la API (registrar un evento de auditoría y rechazar con 403 una acción no permitida). La separación entre admisión (UC5) y registro clínico (UC8) aplica el mínimo privilegio: la admisionista no ve diagnósticos.

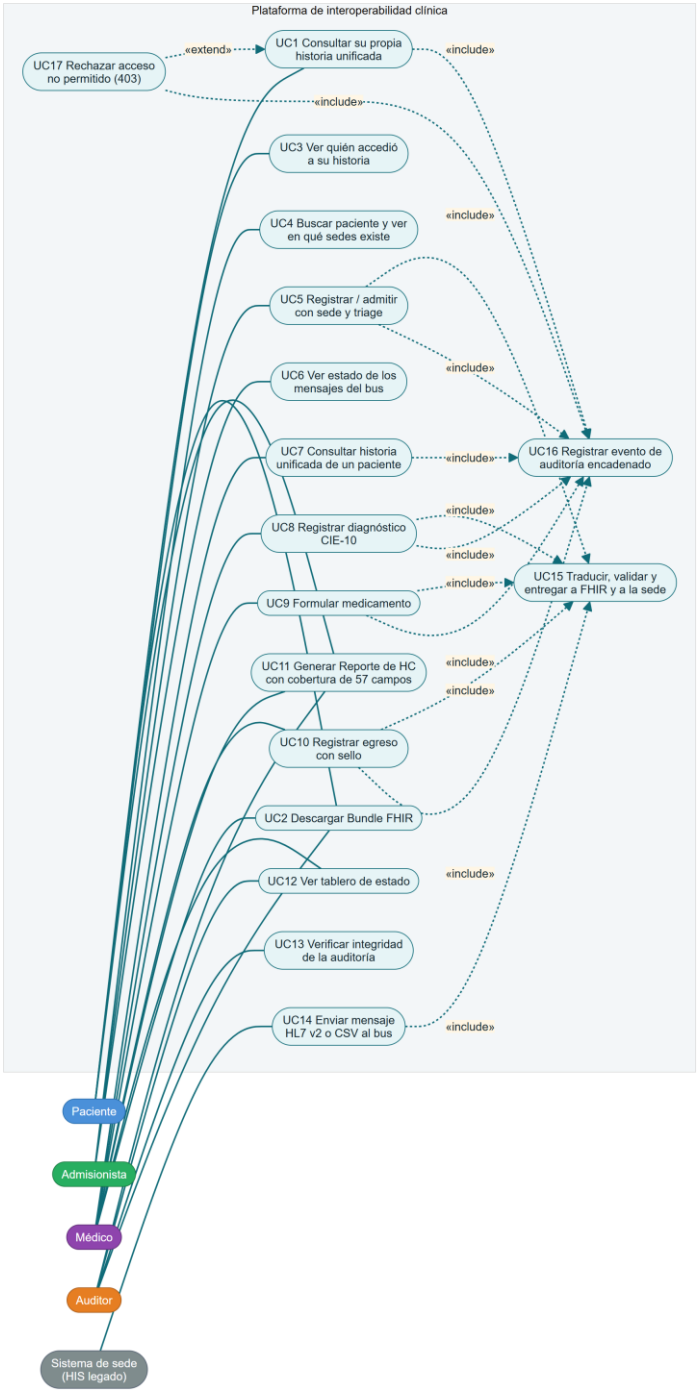


Figura 2. Diagrama de casos de uso: actores, límite del sistema y relaciones «include» y «extend». Fuente: `docs/fase1/diagramas/01_casos_uso.md`.

Diagrama de Secuencias

El diagrama de secuencia (Figura 3) sigue un mensaje HL7 v2 desde el sistema de la sede hasta HAPI FHIR y la base de la sede, con los caminos alternativos de duplicado, rechazo,

error de datos (directo a MUERTO) y reintento en segundo plano. Termina con la consulta del médico, que reúne las tres sedes en paralelo y deja su evento en la cadena de auditoría. El orden HAPI primero y sede después es seguro porque los dos pasos son idempotentes. Por su tamaño se presenta en una página horizontal.

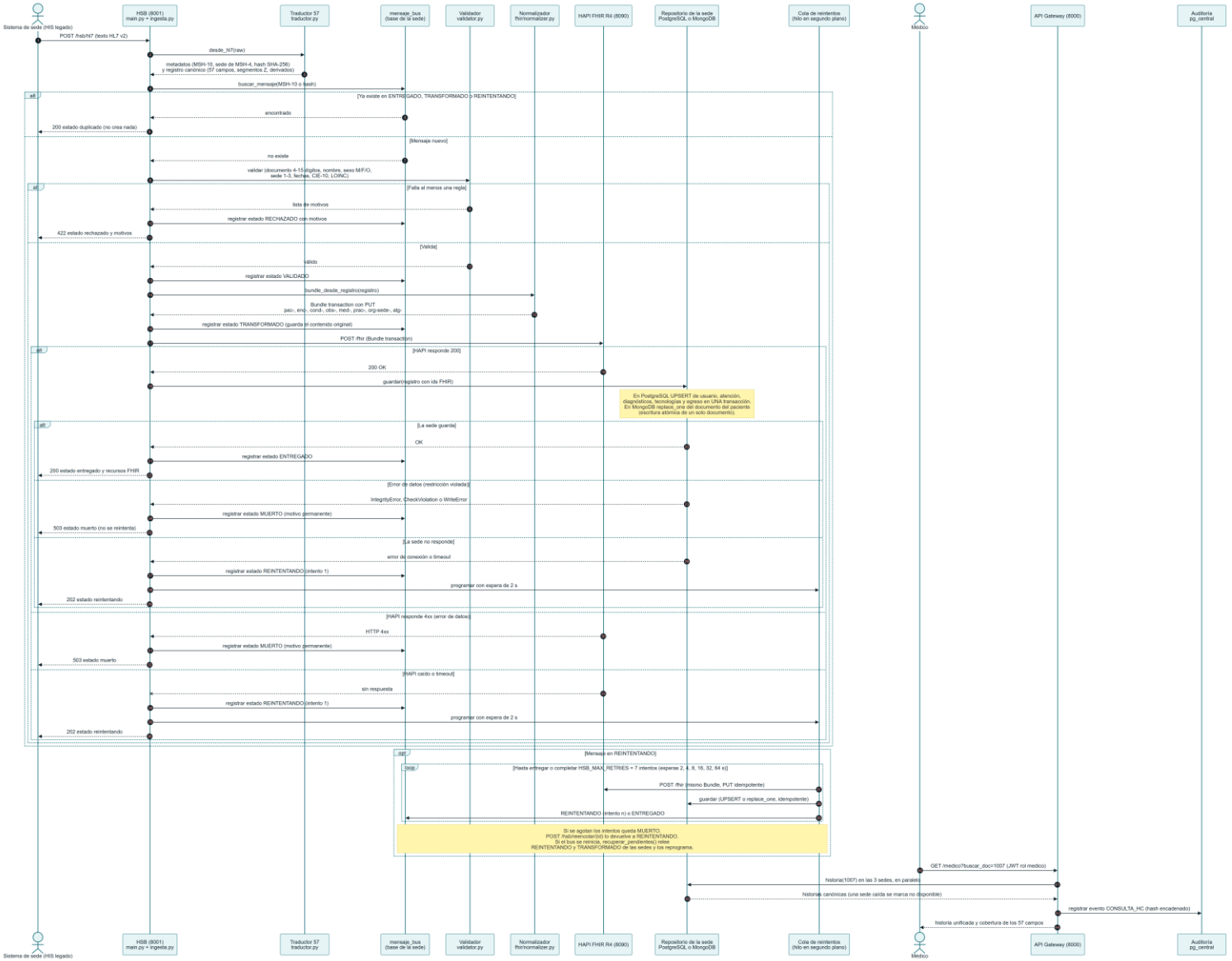


Figura 3. Diagrama de secuencia del procesamiento de un mensaje HL7 v2 en el HSB y de la consulta unificada. Fuente: `docs/fase1/diagramas/02_secuencia_hsb.md`.

Diagrama de Clases

El diagrama de clases (Figura 4) tiene tres grupos: el dominio clínico de la Resolución 866 (`Usuario`, `Atencion`, `Diagnostico`, `TecnologiaSalud`, `Egreso`, `ProfesionalSalud`, `Sede`), el traductor (`Campo`, `RegistroCanonico`) y la infraestructura (`MensajeBus`, `EventoAuditoria`, `UsuarioSistema` y la jerarquía `RepositorioSede` ← `RepositorioPostgres` / `RepositorioMongo`). `Usuario` es el paciente, como lo llama la resolución; quien inicia sesión es `UsuarioSistema`. Los diagnósticos 33 a 39 se modelan como filas con un rol (INGRESO, EGRESO, RELACIONADO).

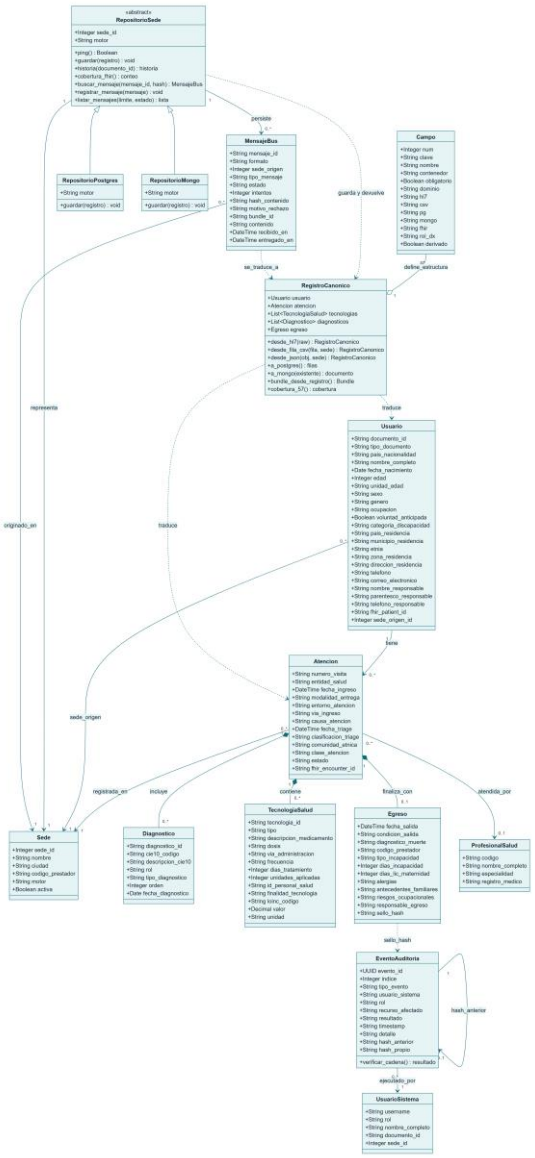


Figura 4. Diagrama de clases del dominio, del traductor y de la infraestructura. Fuente: `docs/fase1/diagramas/03_clases.md`.

Diagrama de Estados

El diagrama de estados (Figura 5) describe el ciclo de vida de un `MensajeBus` con los siete estados que pide la guía: RECIBIDO, VALIDADO, TRANSFORMADO, ENTREGADO, RECHAZADO, REINTENTANDO y MUERTO. El estado se guarda en `mensaje_bus` de la sede del mensaje, por eso sobrevive a un reinicio del bus. "Duplicado" no es un estado: es la respuesta del bus cuando el mensaje ya existe.

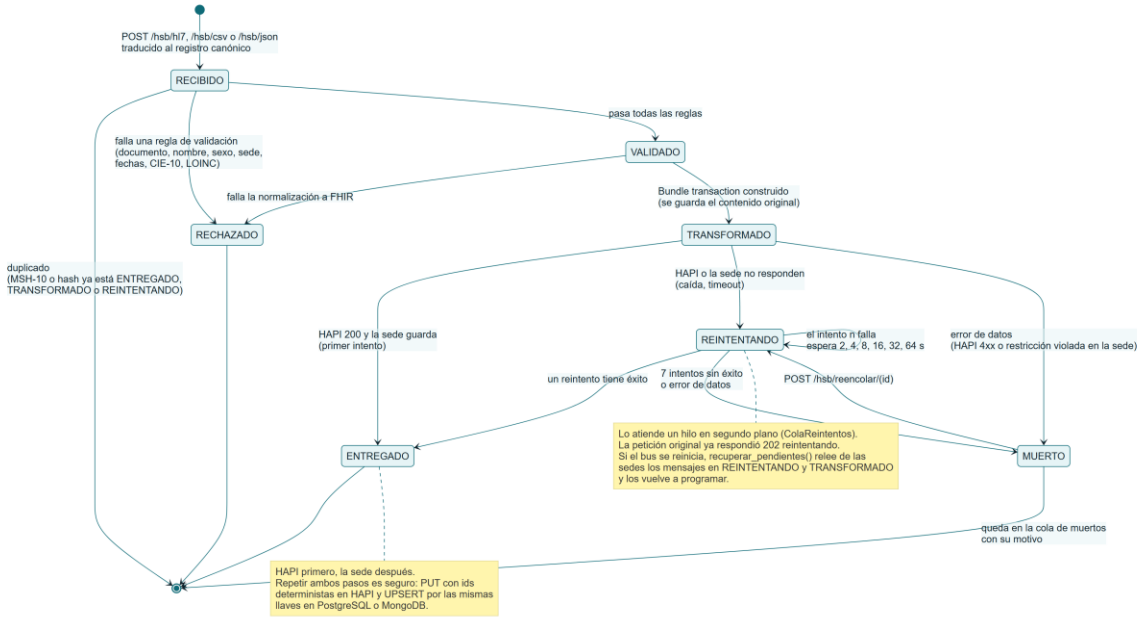


Figura 5. Diagrama de estados del mensaje en el bus. Fuente: `docs/fase1/diagramas/04_estados_mensaje.md`.

Desde	Hacia	Disparador
RECIBIDO	(respuesta `duplicado`)	El identificador o el hash ya existe en ENTREGADO, TRANSFORMADO o REINTENTANDO
RECIBIDO	RECHAZADO	Falla al menos una regla de validación; se guardan todos los motivos
RECIBIDO	VALIDADO	Todas las reglas pasan
VALIDADO	TRANSFORMADO	Bundle FHIR construido
TRANSFORMADO	ENTREGADO	HAPI responde 200/201 y la sede guarda
TRANSFORMADO	REINTENTANDO	HAPI o la sede no responden (caída o tiempo de espera agotado)
TRANSFORMADO / REINTENTANDO	MUERTO	Error de datos: HAPI 4xx o restricción violada en la sede
REINTENTANDO	ENTREGADO / MUERTO	Un reintento tiene éxito / se completan 7 intentos
MUERTO	REINTENTANDO	Un operador llama `POST /hsb/reencolar/{id}`

Tabla 10. Transiciones del diagrama de estados.

Reglas de validación antes de transformar

El validador (`hsb/validator.py`) aplica estas reglas antes de construir el Bundle. Un valor opcional fuera de su dominio no rechaza el mensaje: el traductor lo omite y el bus devuelve una advertencia.

Campo	Regla	Motivo de rechazo
Documento (campo 2)	No vacío, solo dígitos, de 4 a 15	`documento no numérico` / `documento fuera de rango`
Nombre (4)	No vacío	`nombre no puede ser vacío`
Sexo (8)	`M`, `F` u `O` (después de aplicar sinónimos)	`sexo no válido`
Sede	1, 2 o 3 (`MSH-4` o `?sede=N`)	`sede no válida o desconocida`
Fecha de nacimiento (5)	Válida y no futura	`fecha de nacimiento no válida` / `en el futuro`
Fecha de ingreso (17)	Válida si viene	`fecha de ingreso no válida`
Diagnósticos (34, 36–39)	CIE-10 `^[A-Z][0-9]{2}(\.[0-9]{1,4})?$`; en HL7 no se acepta el sistema local `L`; en CSV el texto local se mapea con `05_ref_terminologia.csv`	`diagnóstico sin mapeo a CIE-10`
Laboratorio	LOINC `^[0-9]+-[0-9]+$`	`código LOINC inválido`
Medicamento (25)	Descripción presente	`medicamento sin descripción`

Tabla 11. Reglas de validación del mensaje antes de transformarlo.

PROCESO DE DESARROLLO

El desarrollo siguió las etapas de análisis, diseño, desarrollo, pruebas y despliegue en cada una de las cinco fases del parcial, más una mejora final (v2).

- **Análisis y diseño (Fase 1):** problema, actores y casos de uso, los cuatro diagramas UML, arquitectura con justificación de componentes y tabla de mapeo de campos.
- **Desarrollo por fases:** cada fase partió del código de la anterior (datos y FHIR, bus, seguridad, vistas). En la v2 se reescribieron la capa de datos (57 campos y MongoDB), el bus (cola persistente con trabajador en segundo plano) y la auditoría (persistida), y se agregaron el traductor, el panel de administración, la gestión de cuentas y el generador de datos sintéticos.
- **Correcciones entre fases:** se registraron en la bitácora. Por ejemplo, la auditoría vivía solo en memoria (se pasó a `pg_central`), los reintentos bloqueaban la petición y terminaban en MUERTO antes de que HAPI volviera (cola en segundo plano con esperas hasta 64 s) y reentregar un mensaje duplicaba diagnósticos (identificadores deterministas y UPSERT).
- **Despliegue:** Docker Compose con un solo comando; las imágenes del HSB y de la API se construyen desde el repositorio. Los puertos se pueden reasignar en `.env`. Se incluyen manifiestos de Kubernetes para Minikube en `infra/k8s/`; este informe no presenta evidencia de su ejecución.

- **CI/CD:** no se configuró un pipeline de integración continua. Las pruebas se corren localmente antes de cada cambio (`python -m pytest tests/ -v`). Queda como trabajo futuro.
- **Control de versiones:** repositorio Git con 16 commits de ambos integrantes en las cinco fases y la etiqueta `v1.0`. Los cambios de la mejora v2 se documentan en la bitácora.
- **Uso declarado de IA:** Antigravity (Google DeepMind) en las fases 1 a 5 y Claude Code (Anthropic) en la mejora v2, con revisión y pruebas de los integrantes (Anexo F).

Pruebas automatizadas

Se escribieron 60 pruebas con `pytest`, organizadas por fase. Todas pasan (`60 passed`, ejecución local del 11 de noviembre de 2026). Además se hicieron las pruebas en vivo sobre Docker que se reportan en Resultados.

Suite	Pruebas	Qué cubre
`tests/fase2/test_normalizer.py`	8	Recursos FHIR, identificadores deterministas, referencias internas, caso inválido
`tests/fase2/test_traductor_57.py`	10	Catálogo de 57 campos, columnas de `db/schema.sql` frente al catálogo, ida y vuelta MongoDB, CSV, JSON por número de campo
`tests/fase3/test_hsb_integration.py`	11	Los 4 ejemplos oficiales, idempotencia HL7 y CSV, reintento, muerto y reencolar, error de datos, sede caída
`tests/fase4/test_security_resilience.py`	8	Permisos por rol (403 y auditoría), cadena de hash y alteración, resiliencia
`tests/fase4/test_usuarios.py`	7	Cuentas semilla, administrador de sede limitado a su sede, auditor de solo lectura, contraseña temporal y cambio obligatorio, cuenta del paciente al admitir
`tests/fase5/test_oauth_seguridad.py`	16	OAuth 2.0: flujos password, client_credentials y refresh con rotación, errores del RFC 6749, JWKS, introspección y revocación; el bus rechaza tokens sin firma válida, vencidos, de otra audiencia o de otra sede
Total	**60**	

Tabla 12. Pruebas automatizadas por suite.

DISEÑO DE INTERFACES E IMPLEMENTACIÓN

Las vistas son plantillas Jinja2 servidas por la API en `https://localhost:8000`. Comparten un marco común (barra superior con la red de IPS, el usuario y su rol) y solo muestran las opciones que el rol puede usar; aun así, los permisos se validan en la API y no en la pantalla. Las capturas se tomaron el 11 de noviembre de 2026 sobre la plataforma desplegada con Docker, con datos sintéticos, a 1440 px de ancho; las capturas de página completa se recortaron a la parte relevante (originales en `docs/capturas/`).

Vista	Ruta	Usuario de prueba	Qué muestra
Inicio de sesión	`/login`	todos	Formulario y accesos de demostración por rol
Panel de administración	`/panel`	`admin_plataforma`, administradores de sede, auditor (lectura)	Resumen en vivo, pacientes, usuarios, integración, FHIR y auditoría

Vista	Ruta	Usuario de prueba	Qué muestra
Admisionista	`/admision`	`admision_laura`	Búsqueda en las 3 sedes, posible duplicado, admisión con triage, cuenta del portal, envíos al bus. Sin diagnósticos
Paciente	`/paciente`	`paciente_carlos` o la cuenta creada al admitir	Su historia de las 3 sedes, descarga del Bundle FHIR y accesos a su historia
Médico	`/medico`	`medico_mateo`	Historia unificada; diagnóstico CIE-10, fórmula y egreso con código de verificación
Reporte de HC y estado	`/reporte`	médico, auditor, paciente (solo el suyo)	Reporte por sede, 57 campos con faltantes, sello, impresión, Bundle; pestaña de estado de la plataforma

Tabla 13. Vistas de la plataforma.

Inicio de sesión y diseño adaptable

La pantalla de inicio (Figura 6) muestra el formulario y, por ser un prototipo con datos sintéticos, los usuarios de demostración. La misma pantalla a 390 px de ancho con el tema oscuro del sistema operativo (Figura 7) comprueba el diseño adaptable.

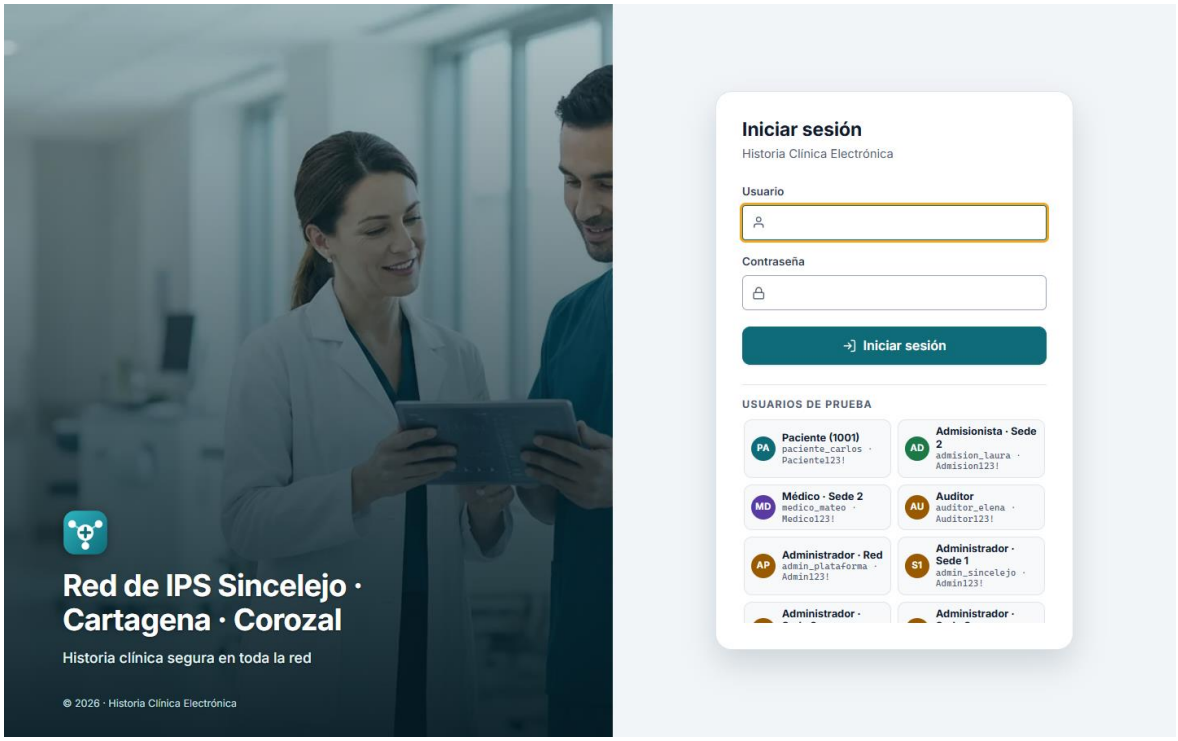


Figura 6. Inicio de sesión con los usuarios de demostración por rol.

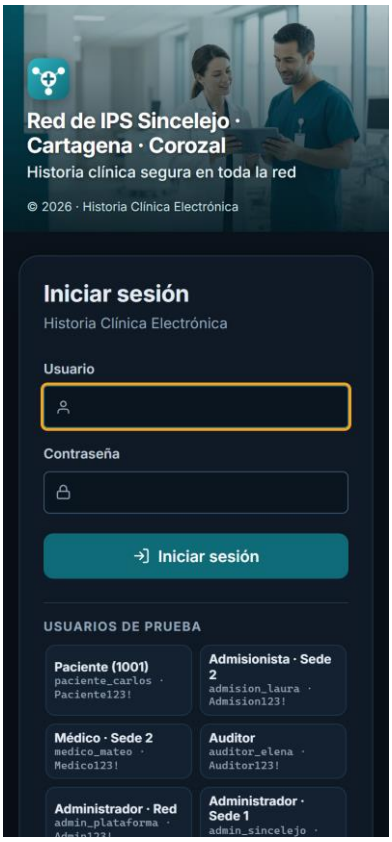


Figura 7. Inicio de sesión a 390 px de ancho con el tema oscuro del sistema.

Panel de administración

El panel se actualiza solo cada 5 segundos y tiene seis secciones. El **Resumen** (Figura 8) reúne indicadores (sedes activas, servidor FHIR, cobertura FHIR, mensajes, latencia media, porcentaje de fallidos, eventos de auditoría), el estado de cada sede con su motor y los últimos mensajes y eventos. **Pacientes** (Figura 9) lista los pacientes de las tres sedes con su sede y si ya están en FHIR, y permite registrar uno nuevo. **Usuarios** (Figura 10) muestra las cuentas con su rol, sede, estado, origen del registro y último acceso, con acciones para desactivar o restablecer la contraseña. **Integración** (Figura 11) muestra los mensajes por estado y las métricas del bus; **FHIR** (Figura 12) los recursos en HAPI por tipo, y **Auditoría** (Figura 13) el estado de la cadena y los últimos eventos con su hash.

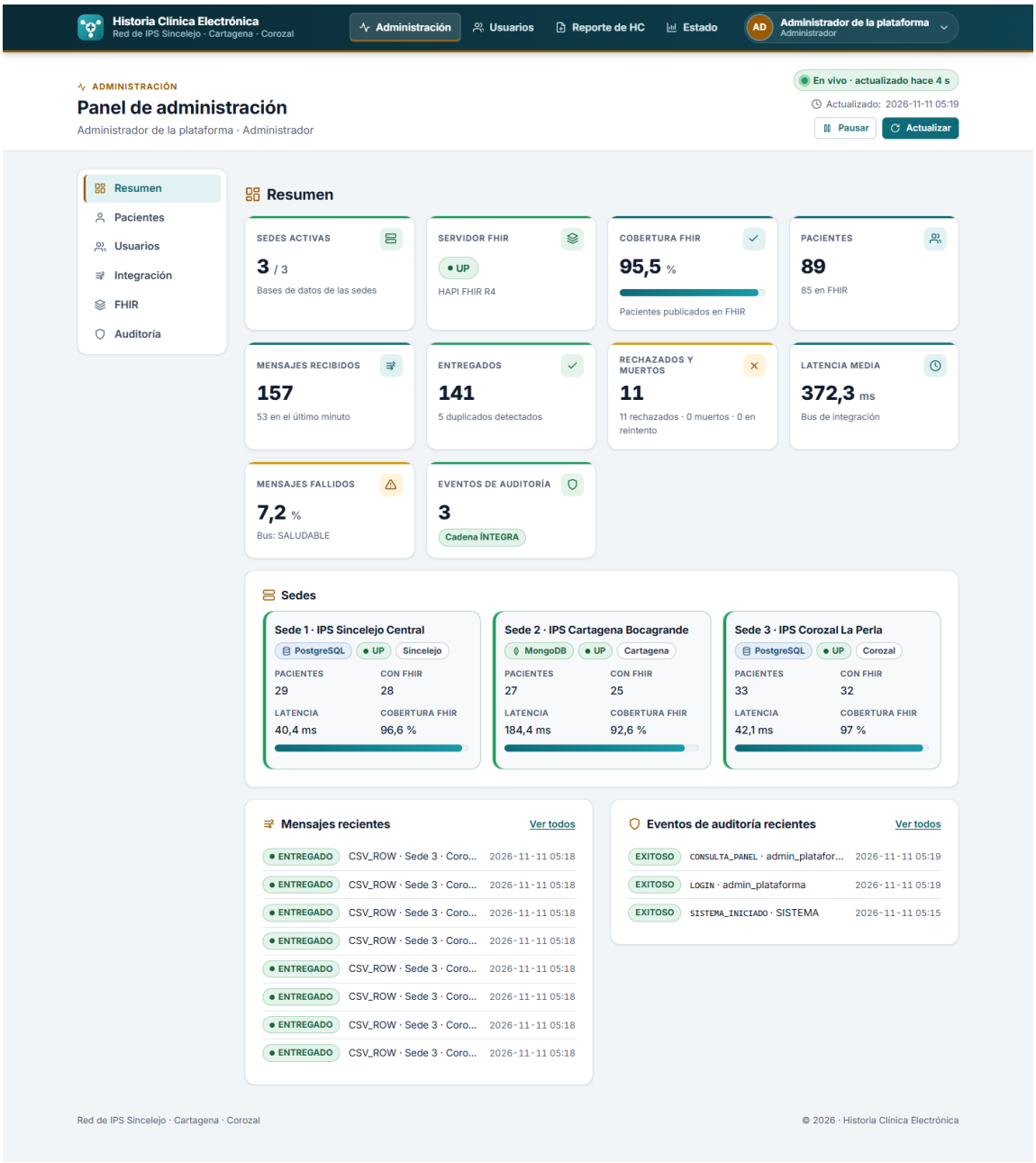


Figura 8. Panel de administración, sección Resumen: indicadores, sedes con su motor, mensajes y eventos recientes.

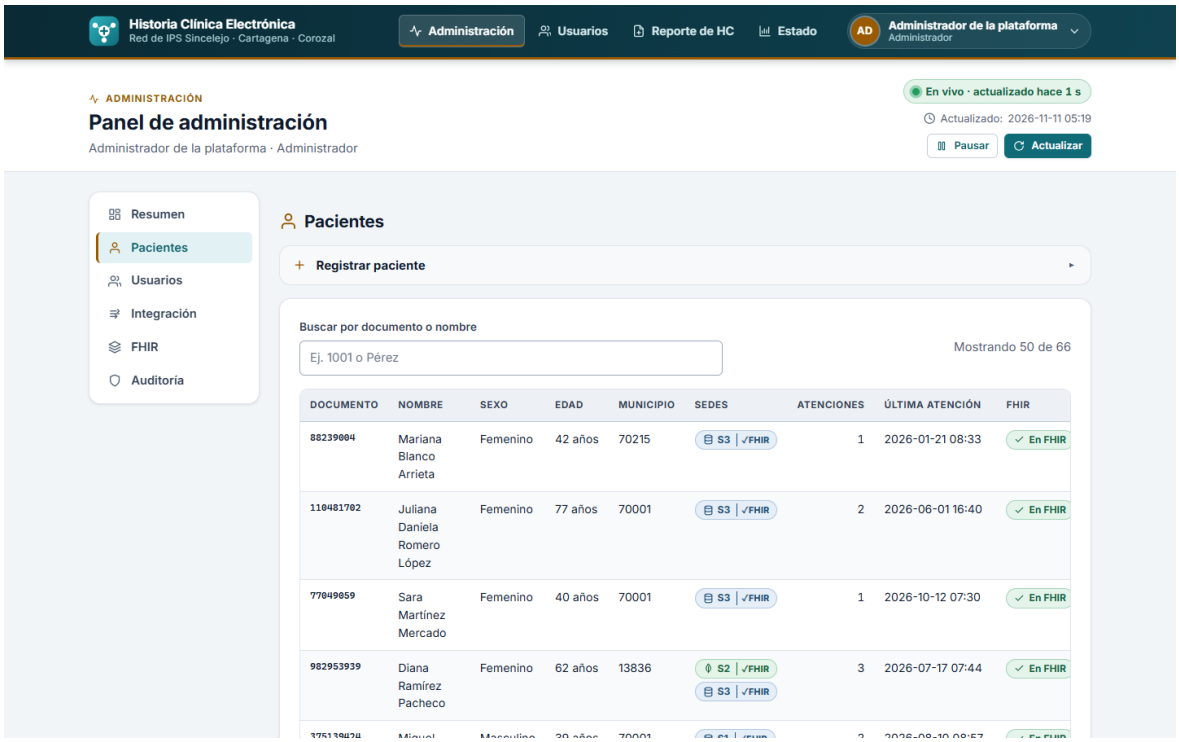


Figura 9. Panel, sección Pacientes: pacientes de las tres sedes con su sede y su estado en FHIR.

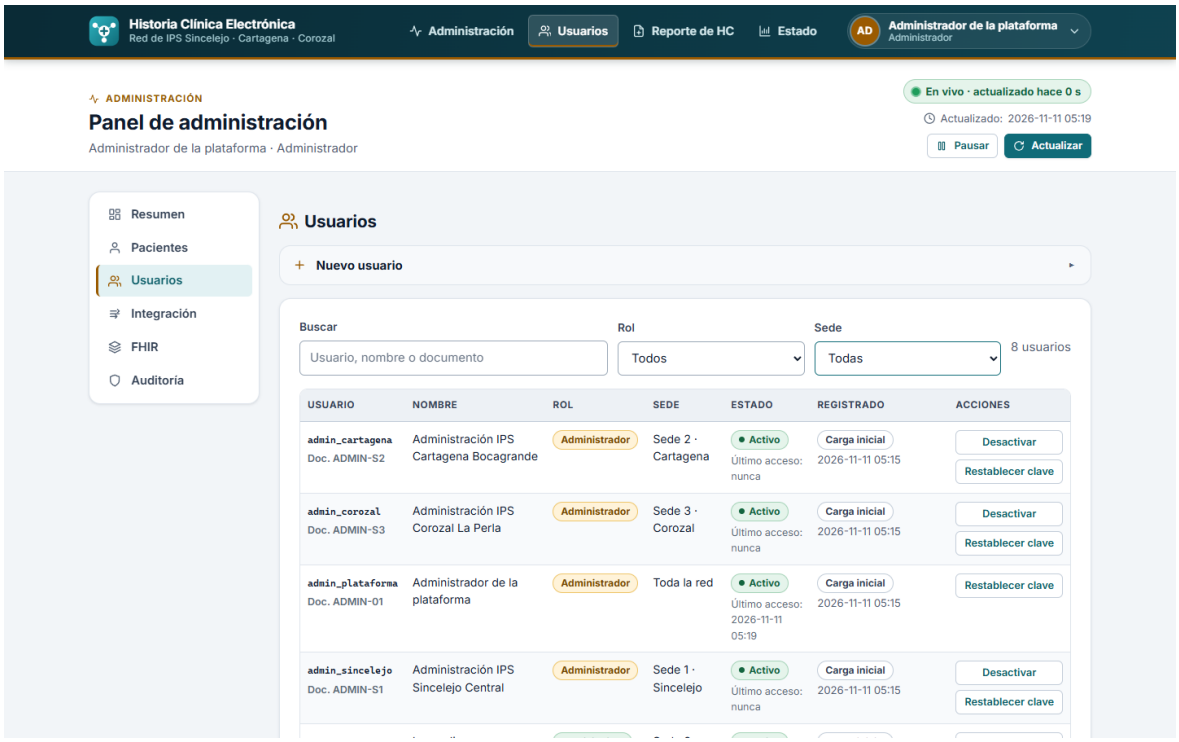


Figura 10. Panel, sección Usuarios: cuentas con rol, sede, estado y origen del registro (carga inicial, panel o admisión).

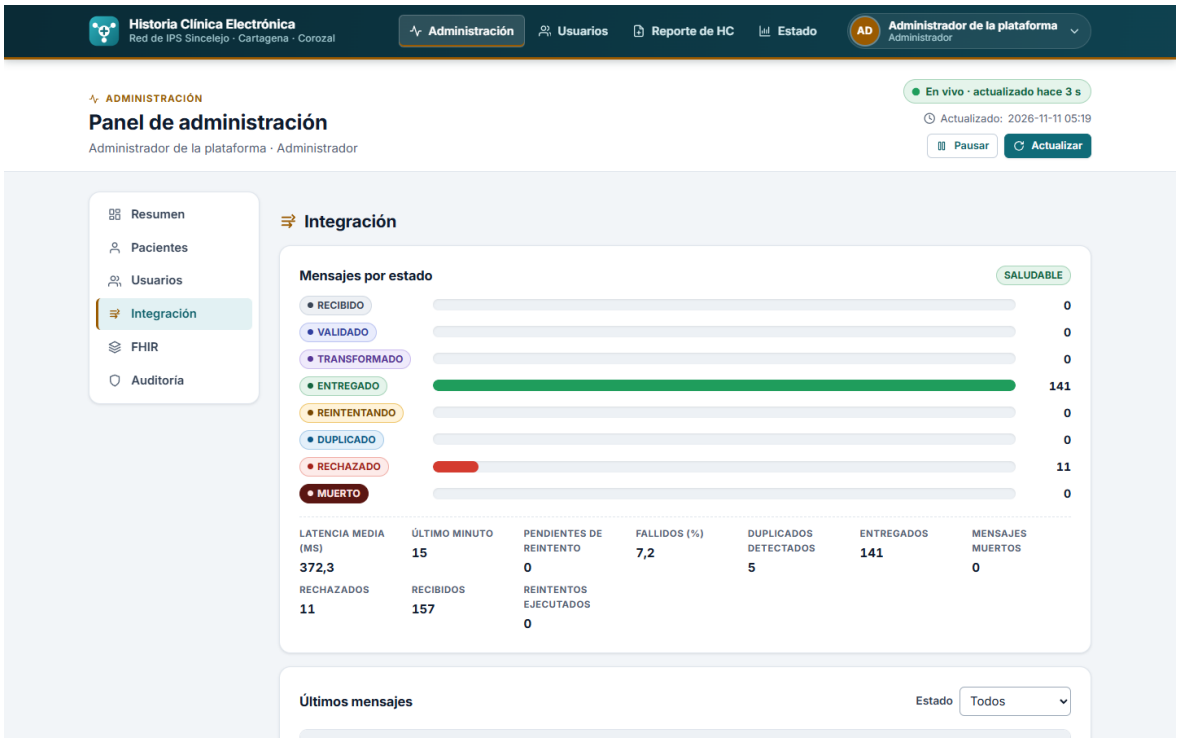


Figura 11. Panel, sección Integración: mensajes del bus por estado y métricas (latencia media, fallidos, duplicados, reintentos).

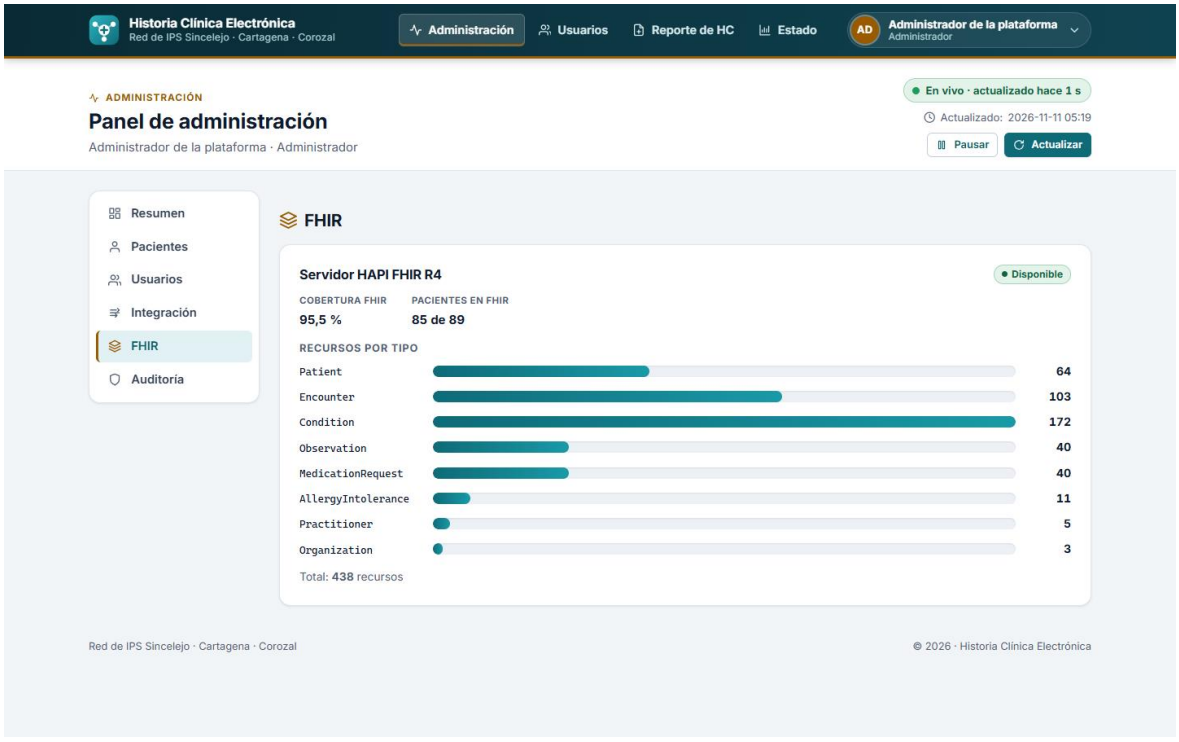


Figura 12. Panel, sección FHIR: cobertura y recursos en HAPI FHIR por tipo.

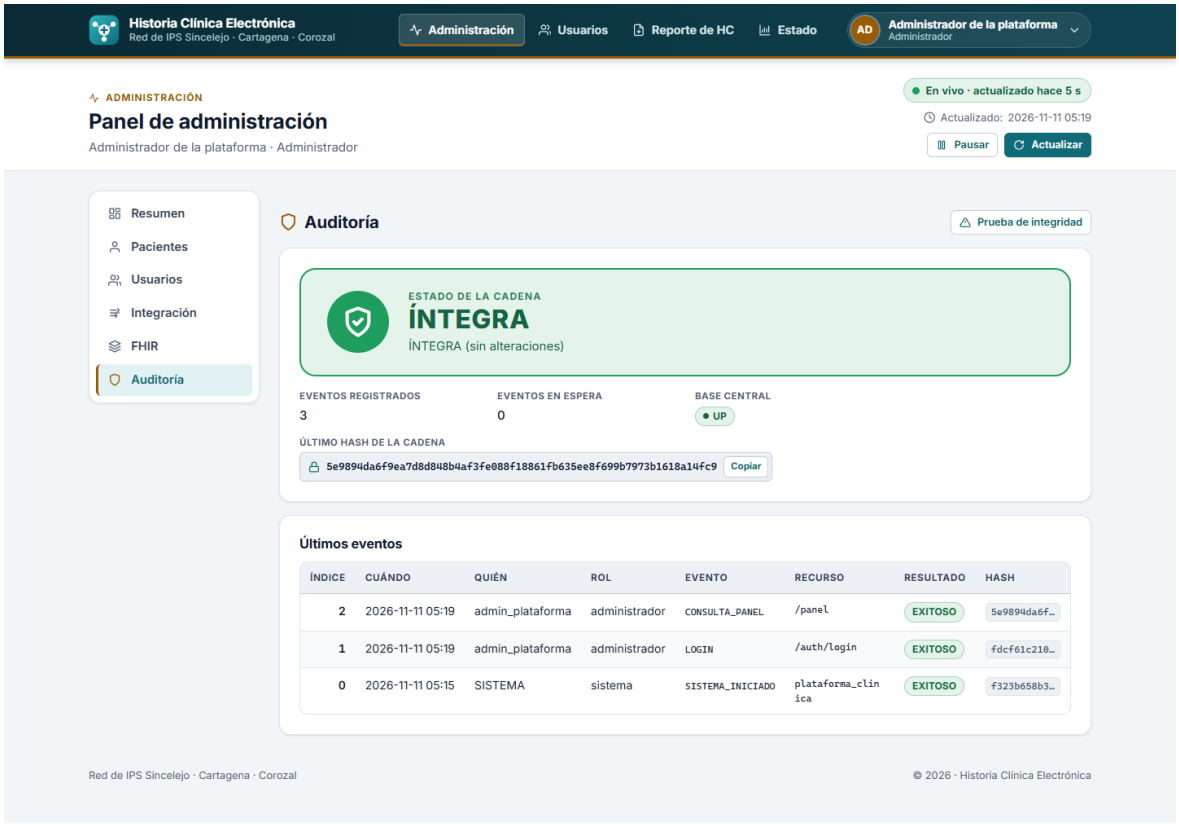


Figura 13. Panel, sección Auditoría: cadena ÍNTEGRA, último hash y últimos eventos.

Admisión y cuenta del paciente

La admisionista busca por documento (Figura 14): la vista indica en qué sedes existe el paciente (con el motor de cada una), advierte un posible duplicado entre sedes y lista las atenciones anteriores sin contenido clínico. Al admitir a un paciente nuevo (Figura 15), el HSB entrega la atención, la vista muestra los recursos FHIR creados y el código de verificación, y aparece la cuenta del portal del paciente con su contraseña temporal, que se muestra una sola vez. En el primer ingreso del paciente el sistema obliga a cambiarla (Figura 16).

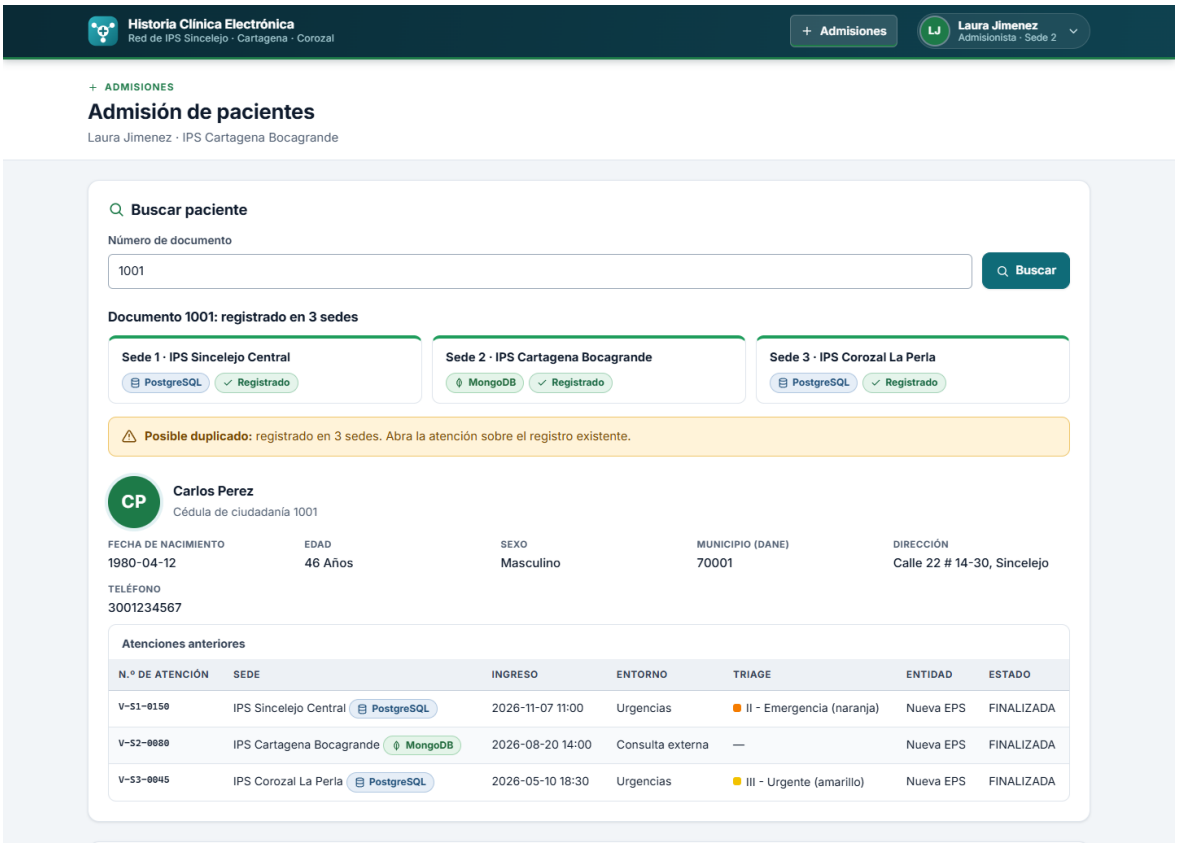


Figura 14. Admisión: el documento 1001 está registrado en las 3 sedes (PostgreSQL, MongoDB, PostgreSQL), aviso de posible duplicado y atenciones sin diagnósticos.

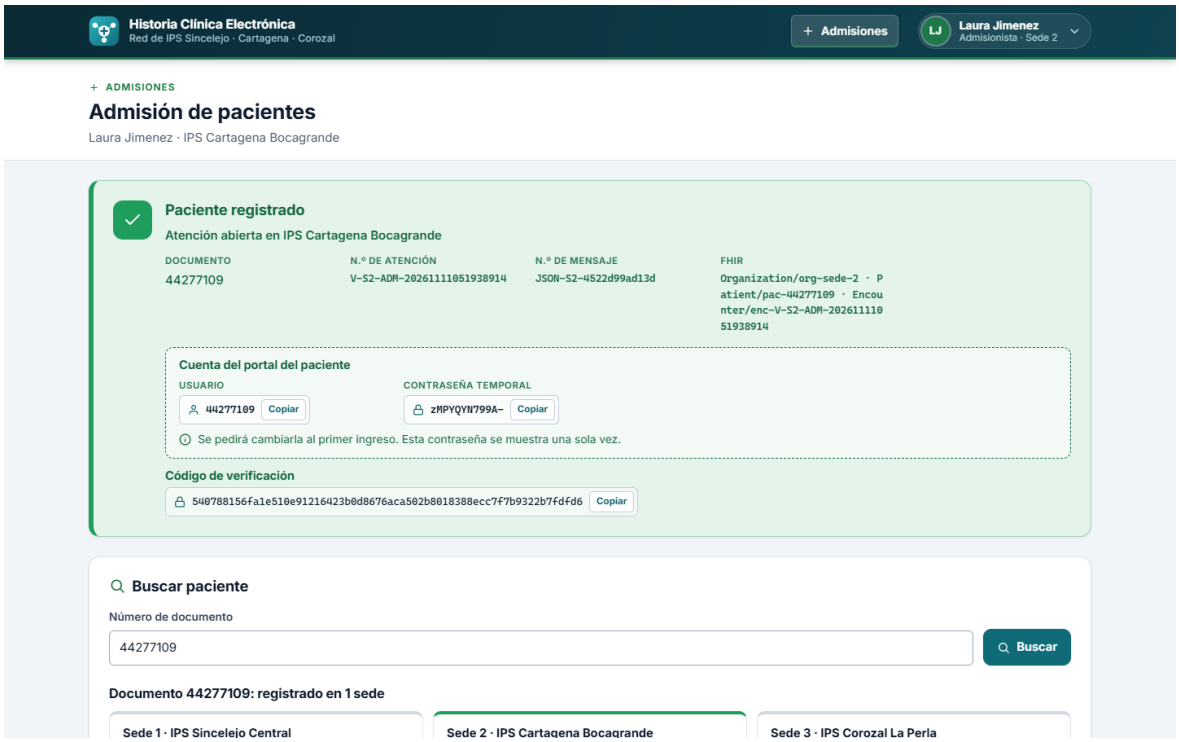


Figura 15. Admisión de un paciente nuevo en la Sede 2: respuesta del bus, recursos FHIR, cuenta del portal con contraseña temporal y código de verificación.

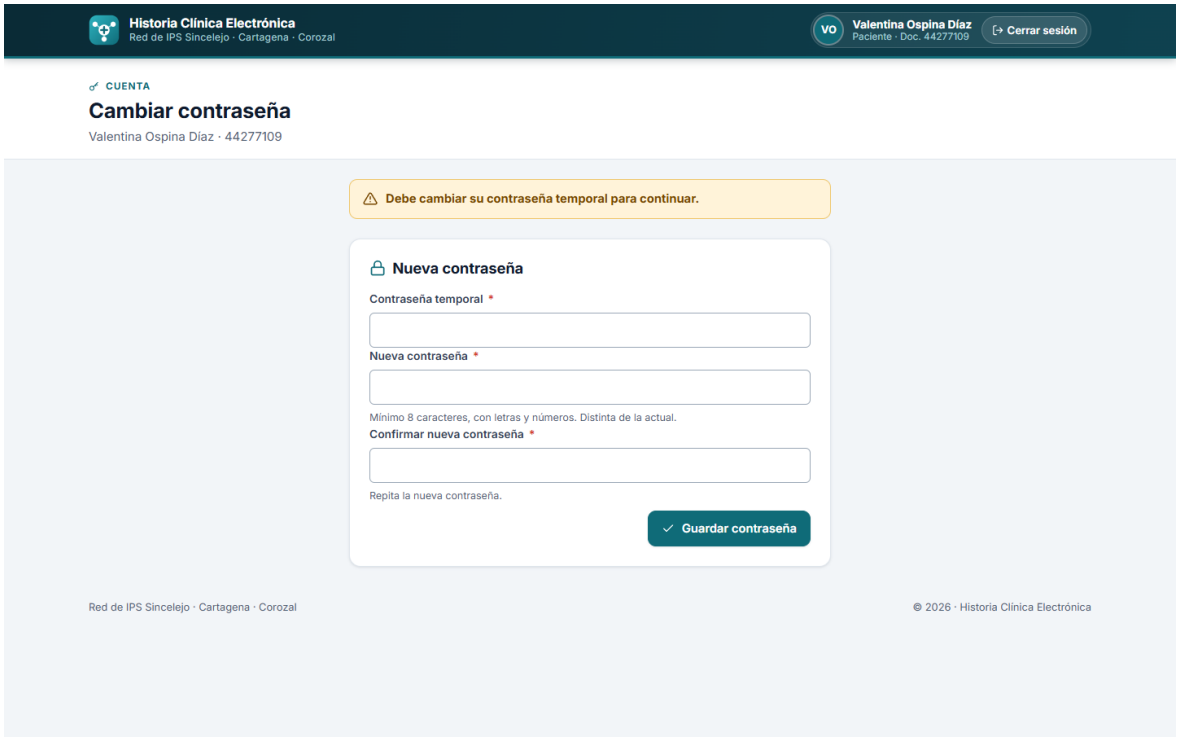


Figura 16. Cambio de contraseña obligatorio en el primer ingreso del paciente.

Portal del paciente

El paciente ve sus datos, su historia ordenada por fecha con la sede de cada atención, un botón para descargar su Bundle FHIR y la lista de accesos a su historia: quién, con qué rol, qué hizo, el resultado y el código de verificación de cada evento (Figura 17). Esta lista responde al derecho de acceso de la Ley 1581 de 2012.

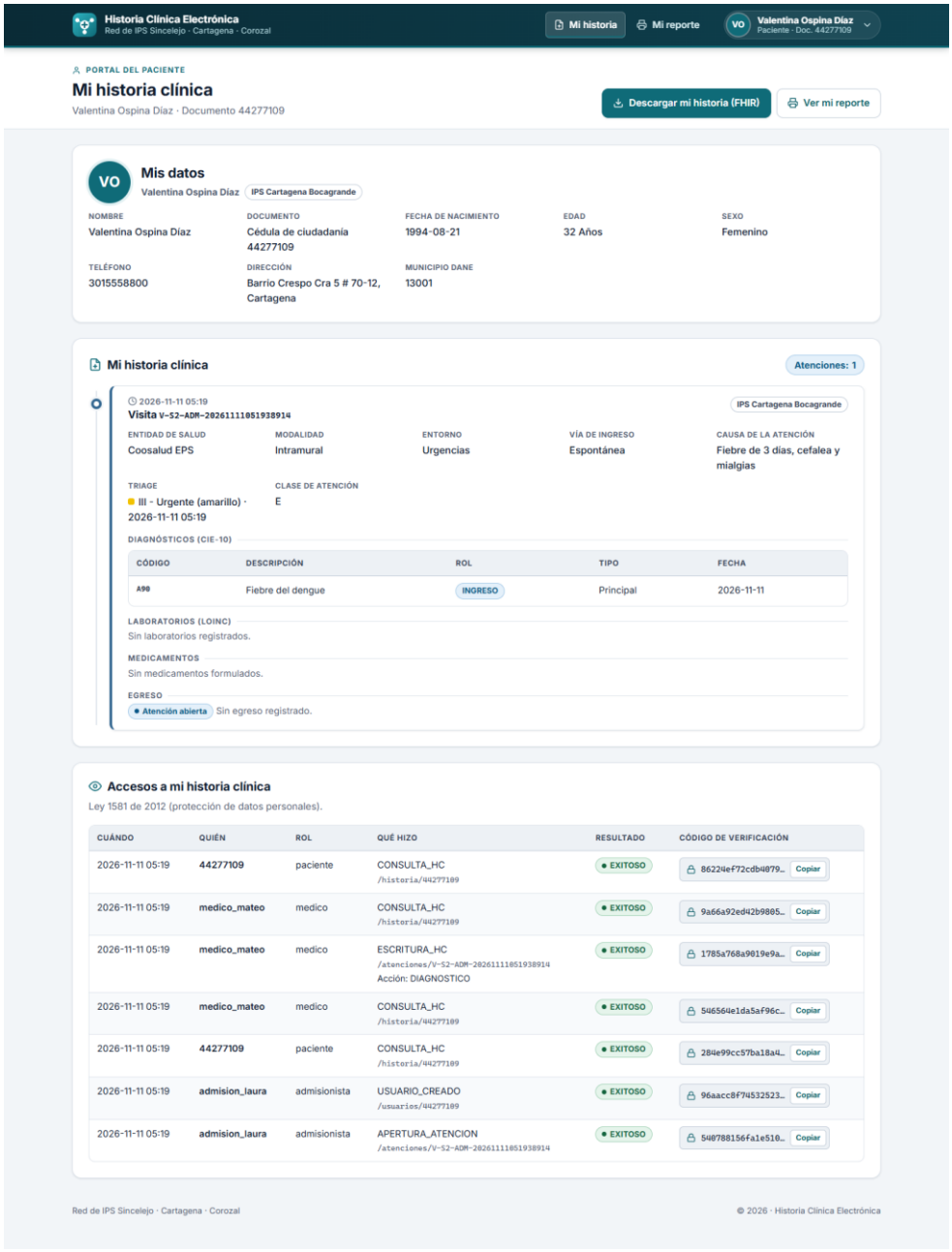


Figura 17. Portal del paciente: datos, historia clínica y accesos a su historia con su código de verificación.

Consulta médica

El médico busca la historia unificada (Figura 18) y registra diagnóstico CIE-10, fórmula (campos 25 a 32) o egreso (campos 40 a 50). Cada escritura viaja por el HSB y devuelve un código de verificación, que es el hash del evento de auditoría de esa escritura (Figura 19). El profesional se toma del token, no del formulario: el médico no puede registrar a nombre de otro.

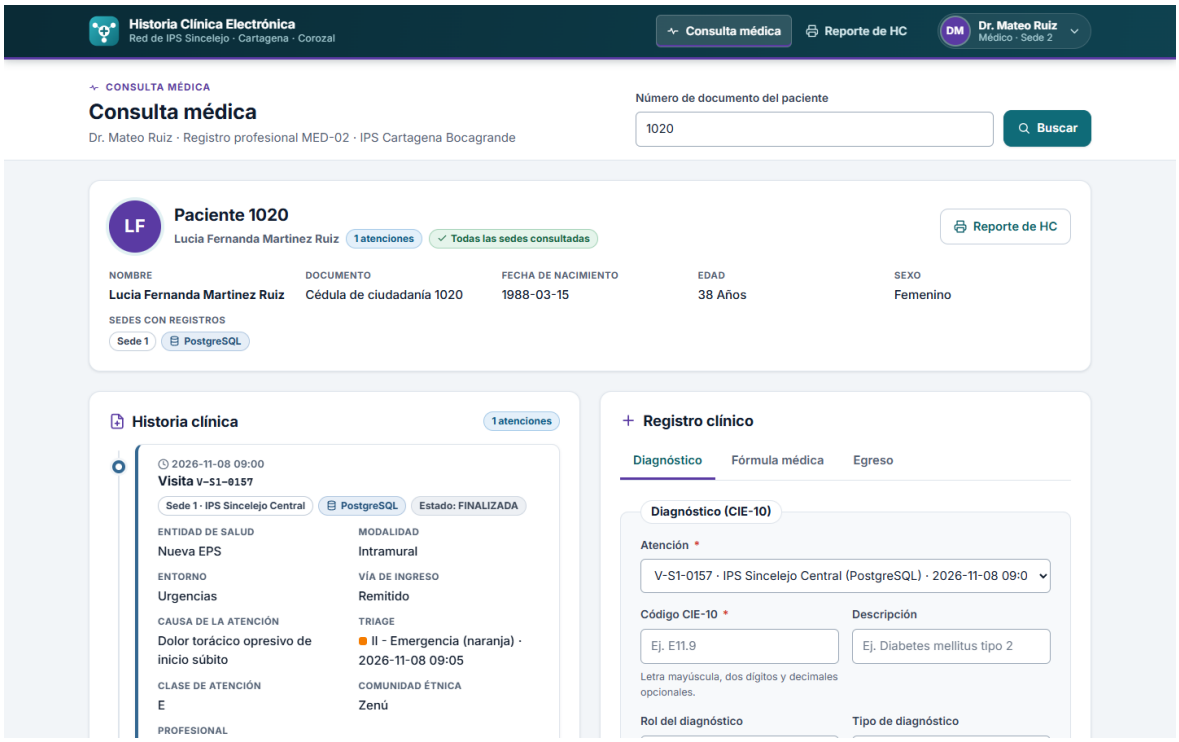


Figura 18. Consulta médica del paciente 1020: historia con la sede y el motor, y formularios de registro clínico.

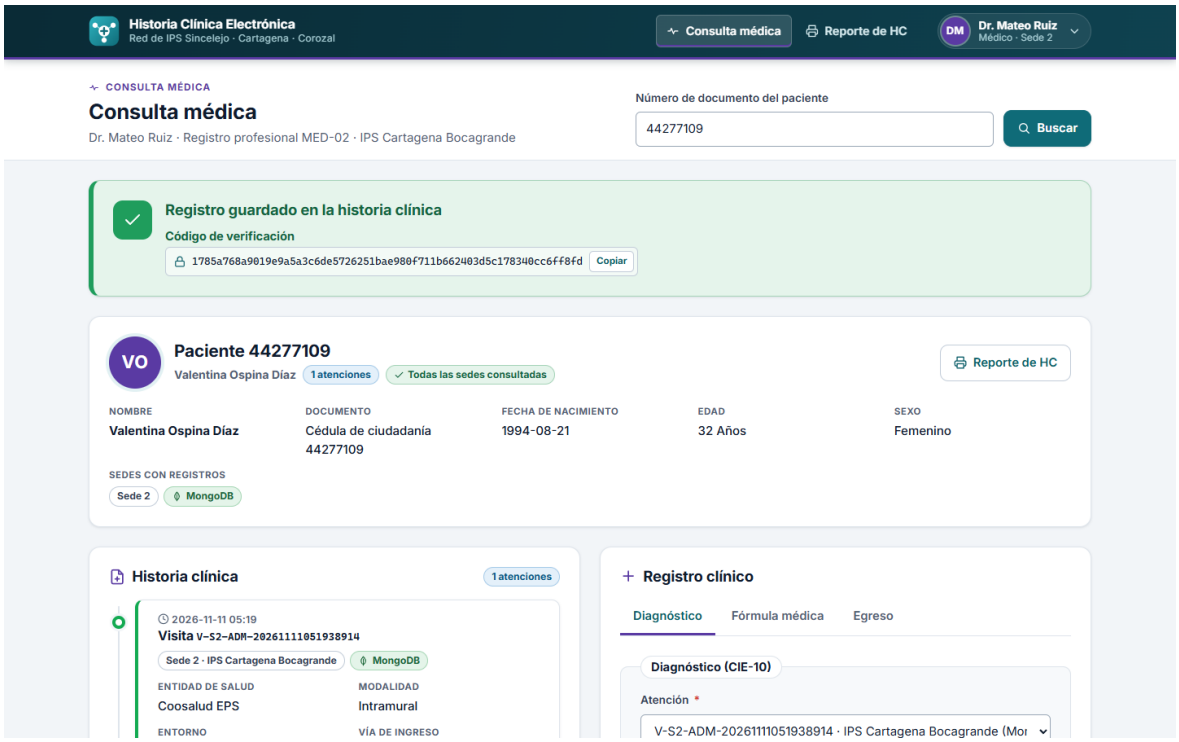


Figura 19. Registro clínico guardado: el médico recibe el código de verificación (sello SHA-256) de la escritura.

Reporte de historia clínica y estado de la plataforma

El Reporte de HC agrupa la historia por sede, indica si el reporte está completo, ofrece imprimir y descargar el Bundle FHIR (Figura 20), y al final muestra la completitud de los 57 campos, la lista de faltantes y el código de verificación del reporte (SHA-256 del contenido). La pestaña Estado de la plataforma (Figura 21) reúne las sedes con su motor y latencia, la cobertura FHIR por sede, las métricas del bus y la cadena de auditoría con su prueba de integridad.

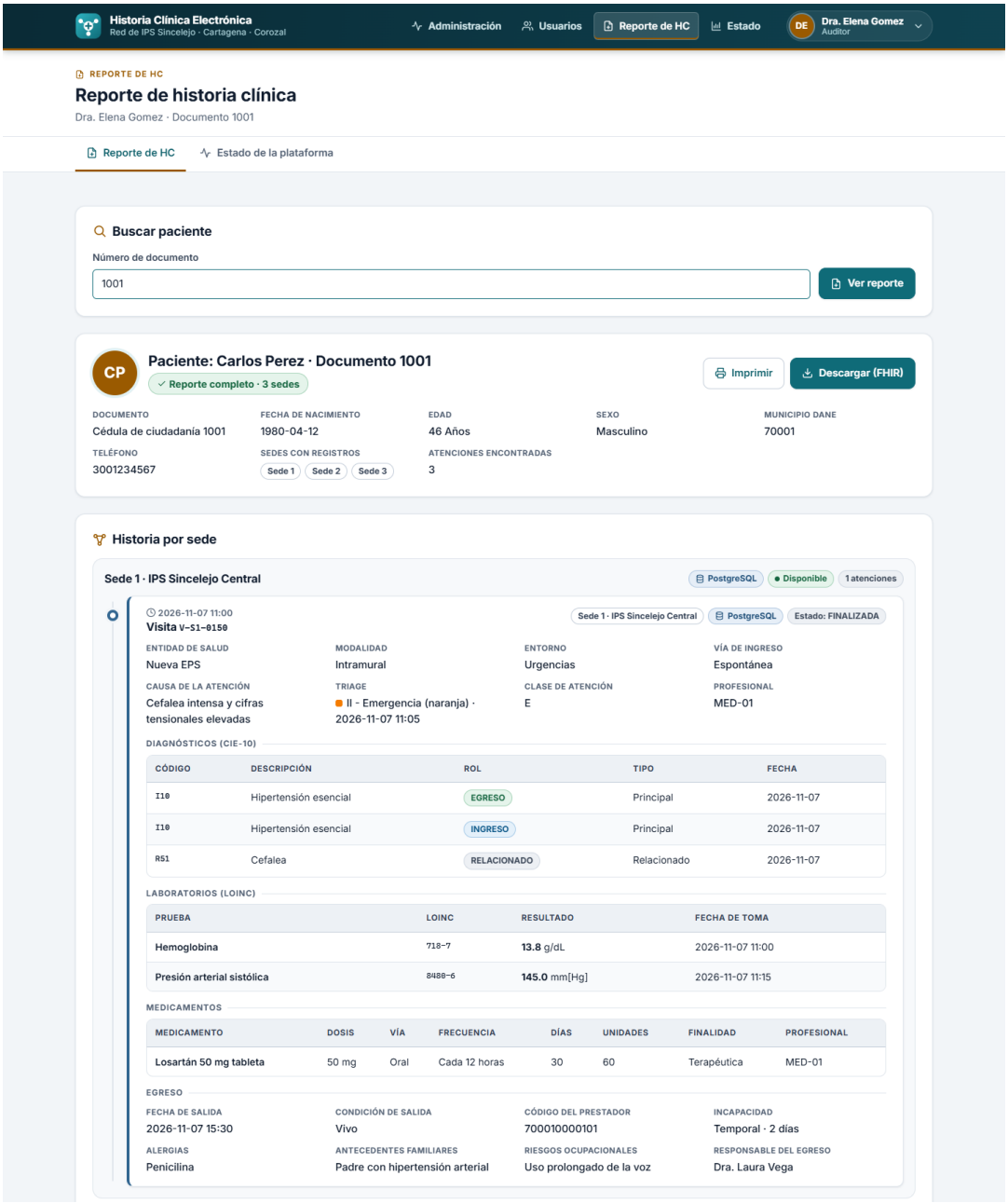


Figura 20. Reporte de HC del paciente 1001 (parte superior): reporte completo de 3 sedes, agrupado por sede.

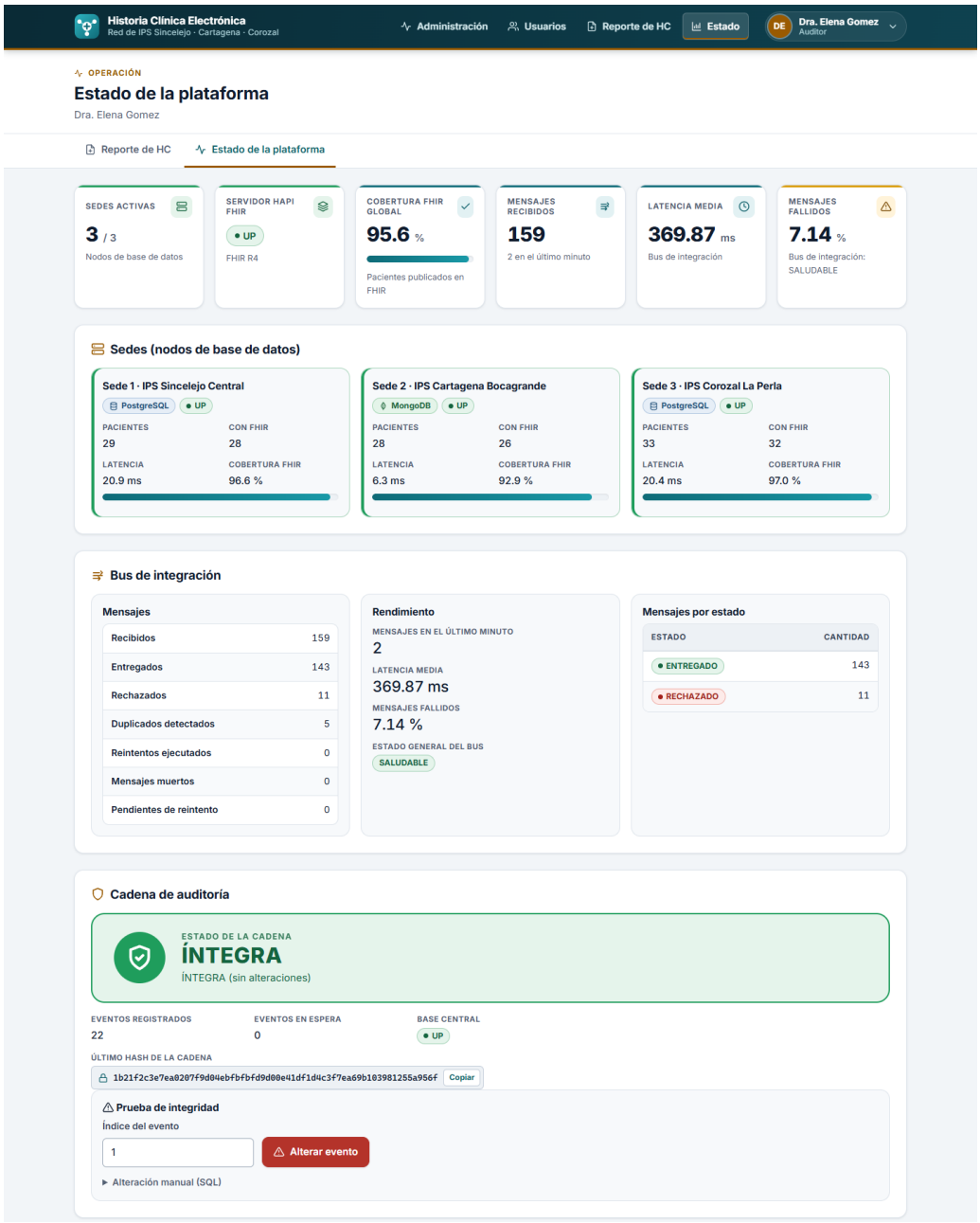


Figura 21. Estado de la plataforma: sedes y su motor, bus de integración y cadena de auditoría.

Experiencia de usuario y accesibilidad

- **Contraste:** texto oscuro sobre fondo claro y estados con colores de contraste suficiente (criterio AA de WCAG como referencia); el tema oscuro respeta la preferencia del sistema.
- **Etiquetas:** cada campo de formulario tiene su `<label>` asociado; los obligatorios se marcan con asterisco y una nota.
- **Teclado:** todas las acciones se pueden hacer con teclado, con foco visible en los campos.

- **Estado en texto:** los estados (ENTREGADO, RECHAZADO, sede no disponible, cadena ÍNTEGRA o ALERTA) se escriben con palabras y no se comunican solo por color.
- **Diseño adaptable:** las vistas se reacomodan hasta 390 px de ancho (Figura 7).
- **Mínimo privilegio visible:** cada rol solo ve sus opciones de menú; el acceso directo a una ruta no permitida muestra un error 403 claro (Figura 22).

Estos criterios se aplicaron y se revisaron manualmente sobre las plantillas finales; no se corrió una herramienta automática de auditoría de accesibilidad.

PROTOTIPO FINAL

El prototipo funcional es la versión v2 de la plataforma. Se probó en una máquina virtual **Kali Linux** en VirtualBox con 3,8 GB de RAM, Docker 28 y Compose v2, con el comando `docker compose down -v && docker compose up -d --build`. HAPI FHIR tarda entre 85 y 95 s en responder después del arranque. Los puertos se pueden reasignar en `.env` si el anfitrión ya los usa. Los pasos de ejecución están en el README y en el Anexo E.

Requisito	Criterio de aceptación	Estado	Evidencia
RF1 Ingesta	HL7 v2 y CSV (y JSON) de las 3 sedes	Cumplido	`/hsb/hl7`, `/hsb/csv?sede=N`, `/hsb/json?sede=N`; ejemplos 01–04 y mensaje de 57 campos
RF2 FHIR R4	Patient, Encounter, Observation y Condition (CIE-10) en Bundle	Cumplido	`fhir/normalizer.py`; Bundle de 1001 con 26 recursos y 0 errores en `$validate`
RF3 HSB	Enrutamiento por sede, transformación, validación, reintentos, idempotencia	Cumplido	`hsb/`; con HAPI apagado, entregado tras 7 intentos sin duplicar
RF4 Datos distribuidos	Un nodo por sede y consulta unificada	Cumplido, con motores distintos	`pg_sede1`, `mongo_sede2`, `pg_sede3`; `db/consulta_unificada.py`
RF5 Seguridad	Token, roles y auditoría encadenada	Cumplido con OAuth 2.0 (Fabric no)	`app/oauth.py`, `app/rbac.py`, `app/audit.py`, `hsb/autorizacion.py`, `pg_central` (Figura 22)
Seguridad v3 (docente)	Autenticación 2.0, seguridad en las bases y en la comunicación entre servidores	Cumplido	OAuth 2.0 RS256; roles de mínimo privilegio; TLS 1.3 y HTTPS con CA propia (Anexo C, `08_seguridad_oauth_bd_tls.txt`)
RF6 Vistas	Paciente, Admisionista, Médico, Reporte de HC con tablero	Cumplido (más panel de administración)	`app/main.py`, `app/templates/`; Figuras de la sección de interfaces
RNF Un comando	`docker compose up` levanta todo	Cumplido	HAPI responde a los 85–95 s
RNF Sede caída	La vista sigue con las otras dos y marca la sede	Cumplido	`docker stop mongo_sede2` (Figura 23)
RNF Observabilidad	`/health` y `/metrics` en cada servicio	Cumplido en HSB y API	HAPI no tiene `/health` propio; su estado lo reportan el HSB y la API

Tabla 14. Matriz de criterios de aceptación frente a RF1–RF6 y requisitos no funcionales.

Evidencias de aceptación

Un usuario con un rol que no tiene permiso para una vista recibe un error 403 y el intento queda en la auditoría (Figura 22).

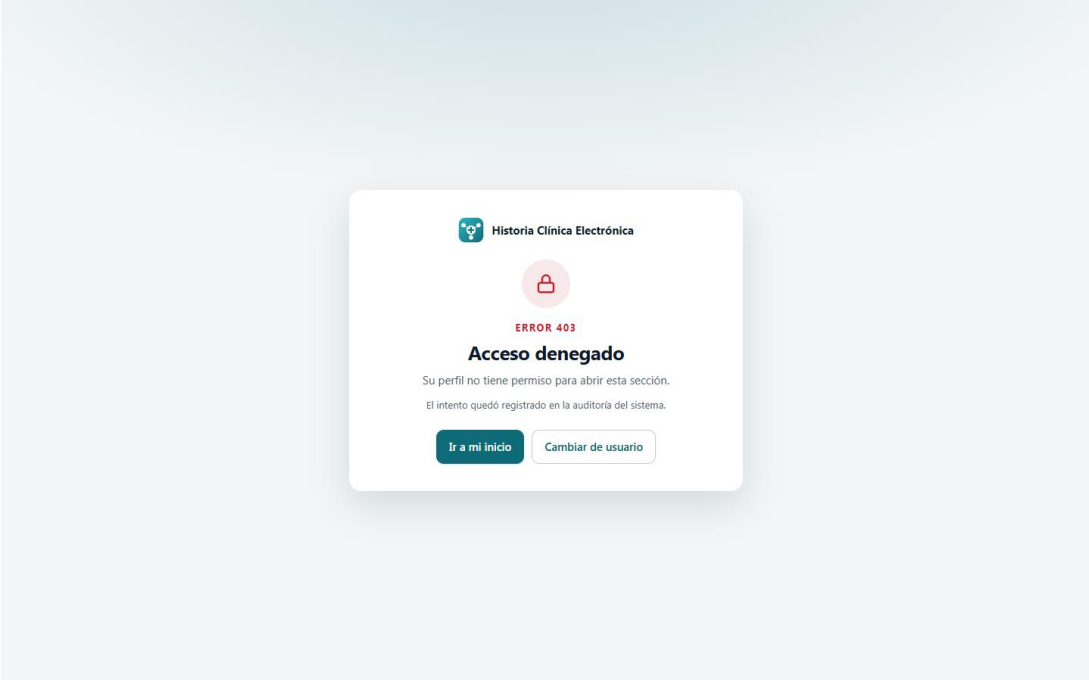


Figura 22. Acceso denegado (403) al abrir una vista no permitida para el rol; el intento queda auditado.

Las siguientes capturas muestran la prueba de caída de la sede MongoDB y la detección de una alteración en la cadena de auditoría.

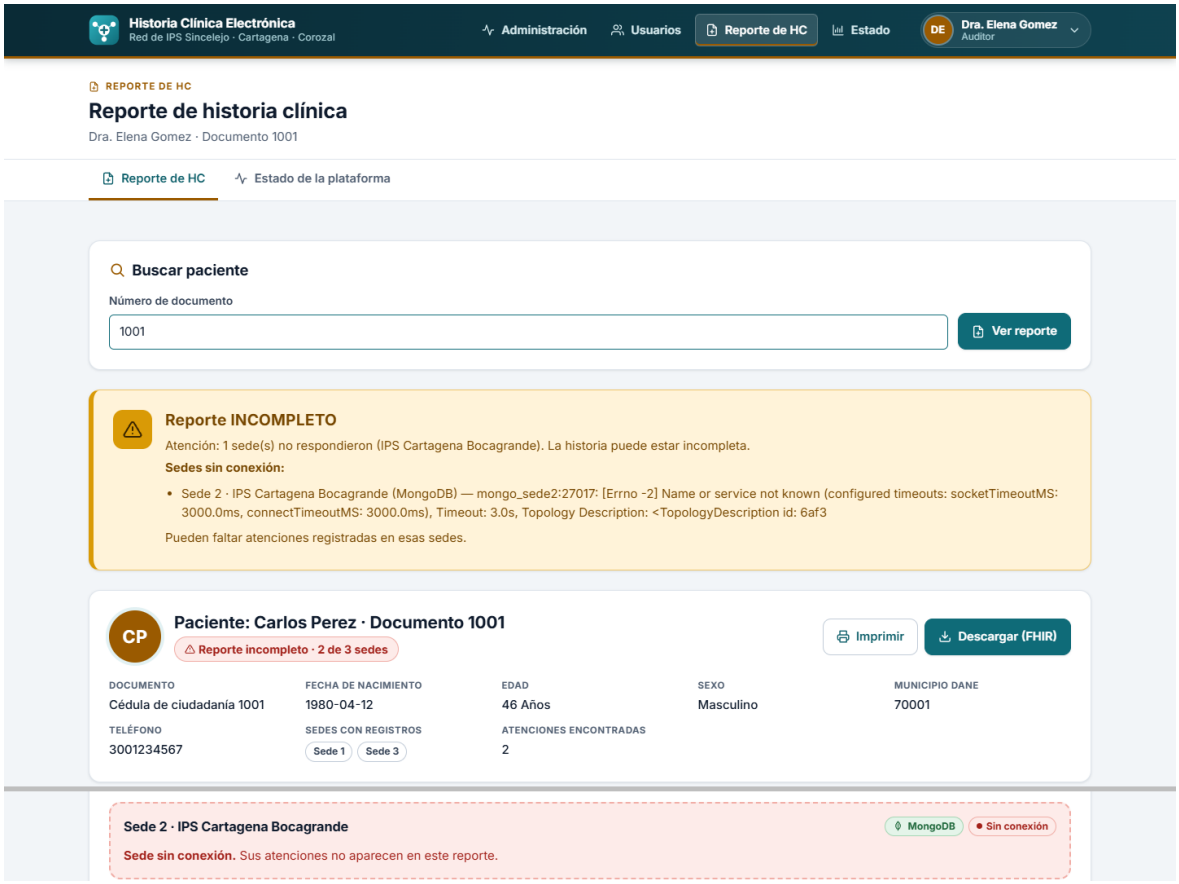


Figura 23. Con `docker stop mongo_sede2`, el Reporte de HC responde con las sedes 1 y 3, marca la Sede 2 como no disponible y el reporte como incompleto.

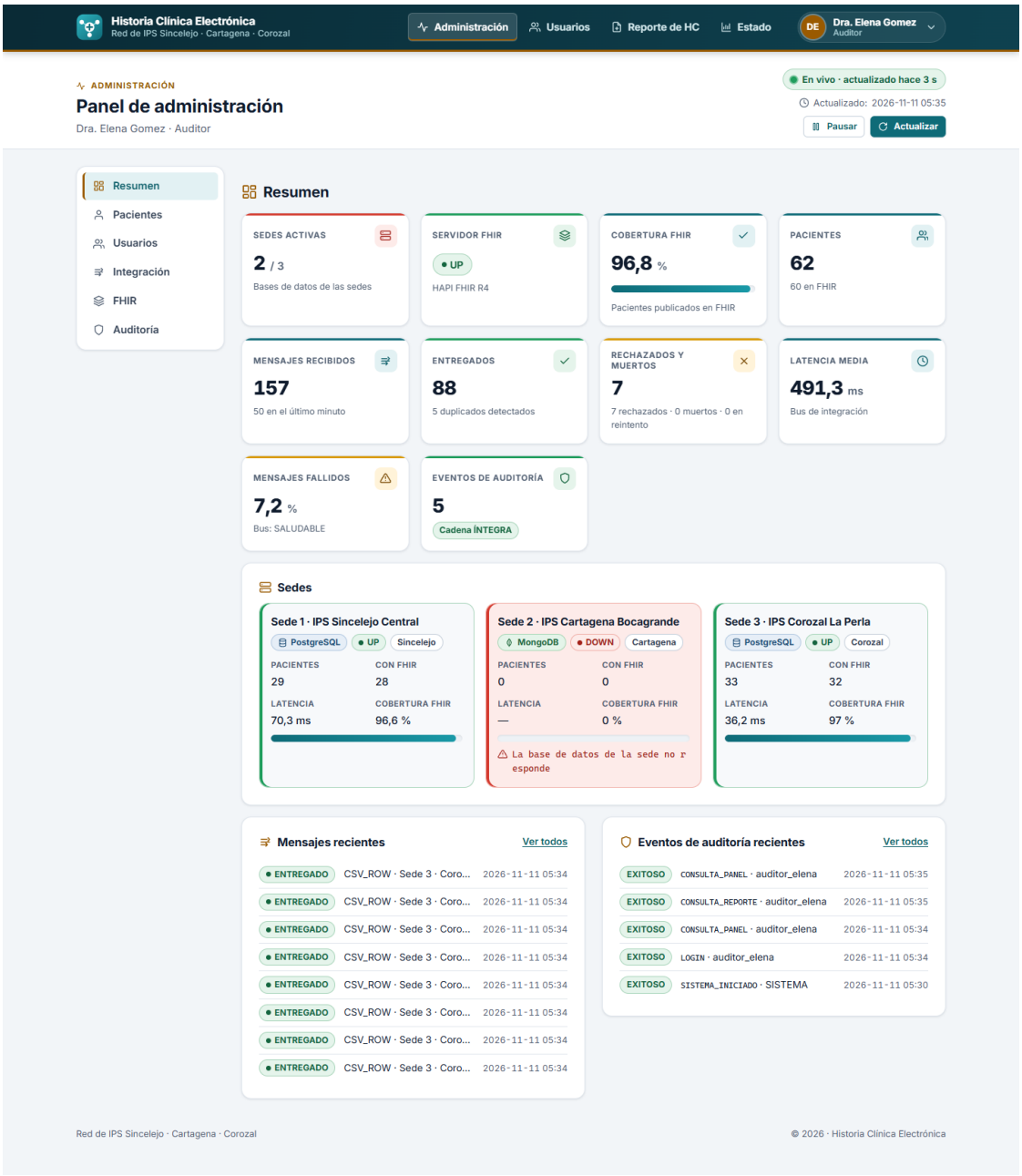


Figura 24. Panel durante la caída: 2 de 3 sedes activas y la Sede 2 (MongoDB) caída.

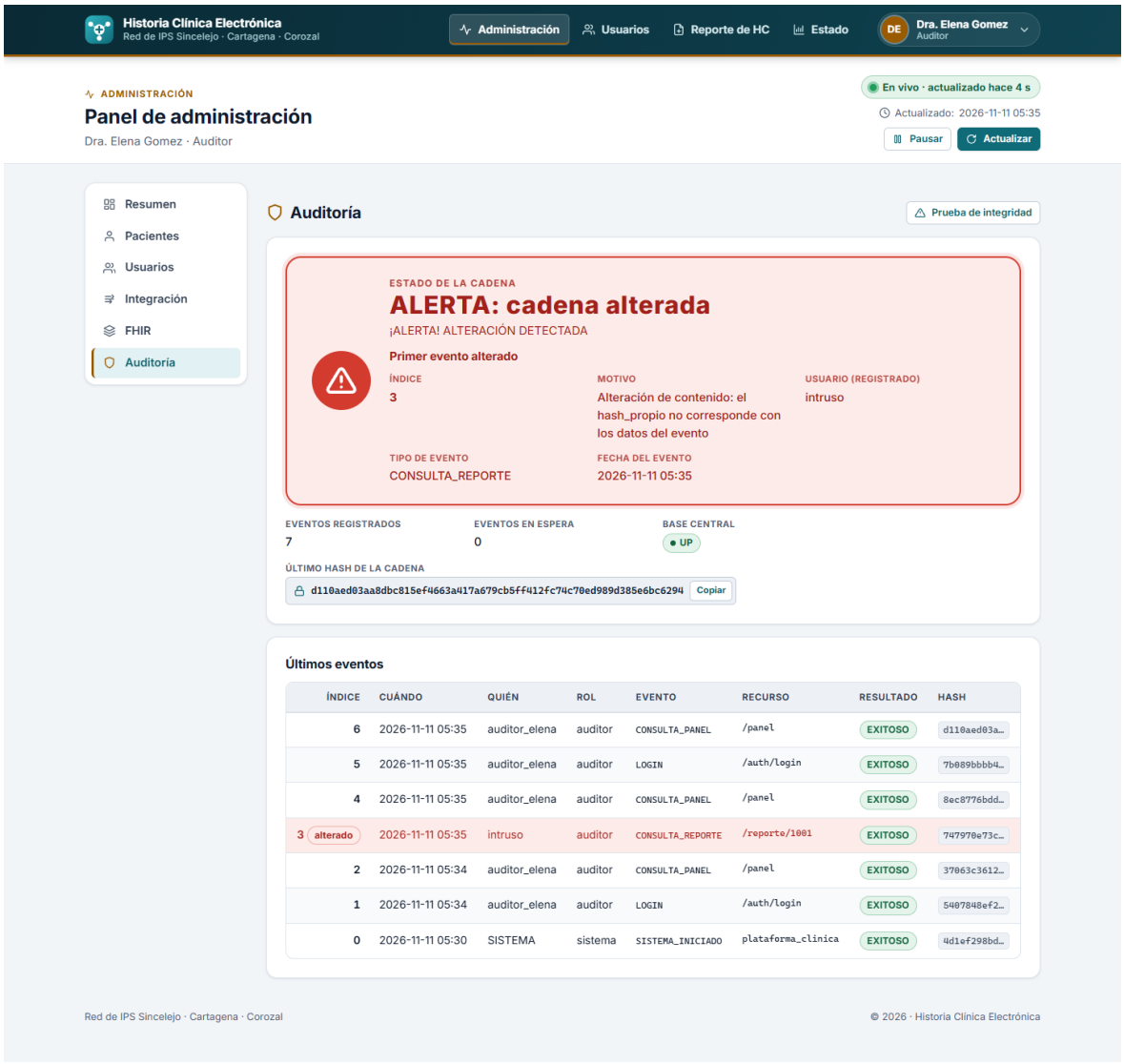


Figura 25. Después de un `UPDATE` manual en `evento_auditoria`, el verificador pasa a ALERTA y señala el primer evento alterado.

RESULTADOS Y DISCUSIÓN

Pruebas funcionales en vivo

Las mediciones se hicieron en Kali Linux (VirtualBox, 3,8 GB de RAM, Docker 28) el 10 y 11 de noviembre de 2026, sobre un arranque limpio y con los mensajes oficiales del docente (`ejemplos/`). Las salidas de consola están en el Anexo C.

Prueba	Resultado medido
Arranque limpio (`down -v && up -d --build`)	Todos los contenedores arriba; HAPI responde a los 85–95 s
Cobertura FHIR antes de enviar mensajes	0 % en las 3 sedes
Ejemplo 01 (ADT^A01, sede 2 MongoDB)	Entregado

Prueba	Resultado medido
Ejemplo 01 reenviado	`duplicado`; exactamente 1 Patient 1007 en HAPI y 1 documento en MongoDB
Ejemplo 02 (ORU^R01, sede 1 PostgreSQL)	Entregado; Observation 718-7 = 13,8 g/dL
Ejemplo 03 (inválido)	Rechazado con 3 motivos
Ejemplo 04 (CSV, sede 3)	3 entregados (1008, 1009, 1010) y 2 rechazados
Mensaje HL7 de 57 campos (`06_adt_a03_57_campos_sede1.hl7`)	11 recursos FHIR; 56 de 57 campos; 100 % de obligatorios (falta solo el 42, diagnóstico de muerte, porque el paciente sale vivo)
Cobertura FHIR por sede después de los ejemplos	Sede 1: 50 % → 67 % (tras el mensaje de 57 campos); sede 2: 33,3 %; sede 3: 75 %
Consulta unificada del paciente 1001 (3 sedes, 2 motores)	52 de 57 campos; 100 % de obligatorios
Bundle de 1001 en HAPI `$validate`	26 recursos (3 Organization, 3 Practitioner, 1 Patient, 3 Encounter, 7 Condition, 5 Observation, 3 MedicationRequest, 1 AllergyIntolerance); 0 errores, 33 advertencias, 46 mensajes informativos
Consulta unificada de 1001 (evidencia del 11 nov.)	`reporte_completo = true`; latencia por sede: 74,8 ms (PostgreSQL), 83,6 ms (MongoDB), 37,9 ms (PostgreSQL)
HAPI apagado al enviar un mensaje	Respuesta `reintentando`; 7 intentos con esperas de 2 a 64 s; entregado solo al volver HAPI, 129,5 s (10 nov.) y 132,0 s (11 nov.) después del primer intento; sin duplicado
HAPI apagado más que la ventana de reintentos	MUERTO tras 7 intentos (Superó 7 intentos); con HAPI de nuevo arriba, `POST /hsb/reencolar` lo entregó en 1 intento (2 607 ms)
Reenvío de 01, 03 y del CSV (11 nov.)	01: `duplicado`, 1 solo Patient 1007 en HAPI; 03: rechazado con 3 motivos; CSV: 3 duplicados y 2 rechazados
`docker stop mongo_sede2`	La consulta responde con sedes 1 y 3, `reporte_completo = false`, `/health` DEGRADED 2/3; se recupera al iniciar el contenedor
`UPDATE evento_auditoria … WHERE indice = 3`	El verificador señala el índice 3: "Alteración de contenido"
Paciente pide la historia de otro paciente	403 y evento `ACCESO_DENEGADO`
ACID en PostgreSQL (`db/demo_acid.sql`)	Violación de `CHECK` en `clasificacion_triage` → rollback completo, sin usuario huérfano
Reinicio del bus con mensajes pendientes	Recuperó 2 mensajes pendientes y los entregó
Pruebas automatizadas	60 de 60 pasan

Tabla 15. Resultados medidos en las pruebas en vivo.

Pruebas de seguridad

Se ejecutaron el 11 de noviembre de 2026 sobre la plataforma desplegada con `desplegar.sh`, conservando los datos existentes (la seguridad se aplicó sobre bases que ya tenían pacientes). La salida completa está en el Anexo C.

Prueba	Resultado medido
Mensaje al bus sin token	401 con `WWW-Authenticate: Bearer`
Mensaje al bus con token `client_credentials`	200 (procesado)
Token de usuario (`aud = api`) usado en el bus	401: la audiencia no corresponde
Sistema de la sede 1 envía un CSV de la sede 3	403 `insufficient_scope`
Secreto de cliente incorrecto	401 `invalid_client`
Refresh token reutilizado después de rotarlo	400 `invalid_grant`
`hsb_escritor` intenta `DELETE` en la sede 1	`permission denied for table usuario`
`api_lector` intenta `UPDATE` en la sede 3	`permission denied`; la lectura sí funciona
`api_central` intenta `UPDATE` y `DELETE` en la auditoría	`permission denied for table evento_auditoria`
MongoDB: `api_lector` inserta, `hsb_escritor` borra	`not authorized` en ambos casos
Conexión sin TLS a PostgreSQL y a MongoDB	Rechazadas (`pg_hba.conf rejects ... no encryption`)
Conexiones de los servicios a las bases	Cada servicio con su rol y `SSL enabled (protocol=TLSv1.3, cipher=TLS_AES_256_GCM_SHA384)`

Prueba	Resultado medido
Certificado del bus	Emitido por la CA de la plataforma; `Verify return code: 0 (ok)`; TLS 1.3
HAPI por HTTPS / por HTTP plano	200 / 400
Verificación sede por sede (`verificar_interoperabilidad.py`)	70 de 70 correctas (63 de interoperabilidad y 7 de OAuth 2.0 y HTTPS)
Colección de Postman con Newman desde el PC anfitrión	58 peticiones y 82 aserciones, todas correctas

Tabla 16. Resultados de las pruebas de seguridad.

Los ejemplos 01 (dos veces), 02, 03 y las 5 filas del CSV son 9 mensajes recibidos: 5 entregados, 1 duplicado y 3 rechazados. Los 3 rechazos son los que el contrato de `ejemplos/LEEME.md` exige rechazar; ningún mensaje válido quedó MUERTO en el flujo normal.

La cobertura de campos se ve en el Reporte de HC: el paciente 1001 reúne 52 de 57 campos desde las tres sedes (Figura 26) y el paciente 1020, con el mensaje de 57 campos, llega a 56 de 57 (Figura 27).

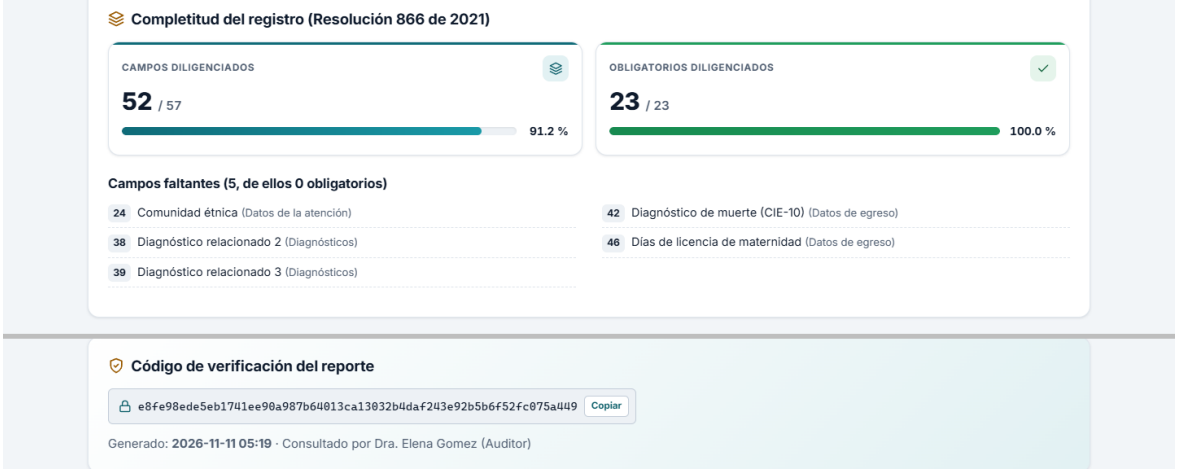


Figura 26. Reporte del paciente 1001 (fragmentos): 52 de 57 campos, 23 de 23 obligatorios, campos faltantes y código de verificación del reporte.

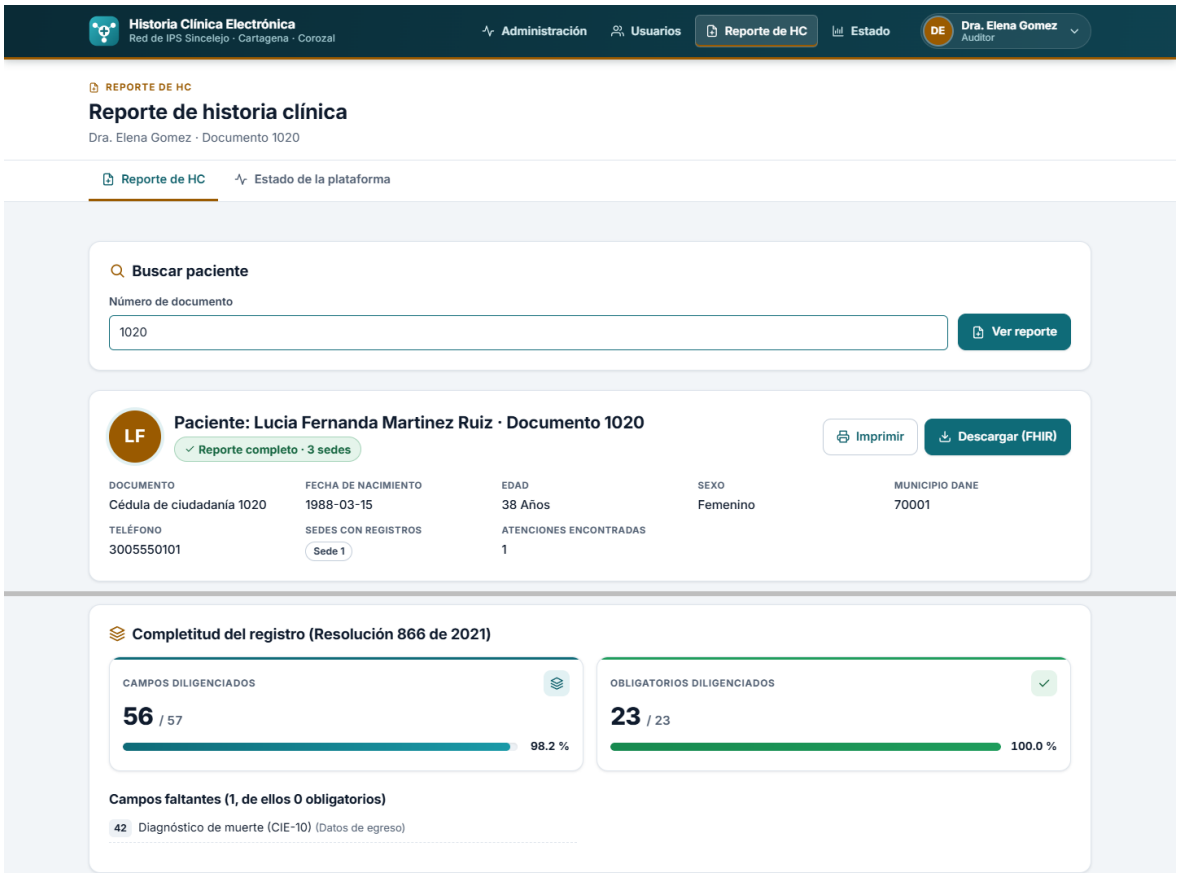


Figura 27. Reporte del paciente 1020 (fragmentos): 56 de 57 campos y 100 % de obligatorios; solo falta el campo 42 (diagnóstico de muerte).

Indicadores operativos del bus

La guía pide reportar latencia, mensajes por minuto, porcentaje de fallidos y cobertura. Para las capturas se cargaron los ejemplos oficiales y el generador de datos sintéticos con 60 pacientes: 148 mensajes, de los que 135 se entregaron, 8 se rechazaron a propósito y 5 eran reenvíos (duplicados). La tabla recoge dos lecturas del 11 de noviembre de 2026: el panel a las 05:19 (Figura 8) y el endpoint `/metrics` del HSB a las 05:22 (Anexo C), después de reenviar los ejemplos 01, 03 y el CSV.

Indicador	Panel (05:19)	`/metrics` (05:22)	Lectura
Mensajes recibidos	157	166	La diferencia incluye los reenvíos de 01, 03 y las 5 filas CSV
Entregados	141	143	Los reenvíos no generan entregas nuevas
Rechazados	11	14	Todos son errores introducidos a propósito en los datos de prueba
Duplicados detectados	5	9	Idempotencia: el ejemplo 01 y las 3 filas válidas del CSV
Mensajes muertos	0	0	Ningún mensaje válido quedó MUERTO en el flujo normal
Porcentaje de fallidos	7,2 %	8,92 %	Rechazados sobre mensajes no duplicados (14 de 157)
Latencia media	372,3 ms	369,87 ms	Promedio por mensaje entregado en la sesión

Indicador	Panel (05:19)	`/metrics` (05:22)	Lectura
Mensajes en el último minuto	53	7	Ventana móvil: depende del momento de la lectura
Cobertura FHIR global	95,5 % (85 de 89 pacientes)	—	Por sede en la pestaña de estado: 96,6 %, 92,9 % y 97,0 % (Figura 21)
Recursos en HAPI	438	—	64 Patient, 103 Encounter, 172 Condition, 40 Observation, 40 MedicationRequest, 11 AllergyIntolerance, 5 Practitioner, 3 Organization (Figura 12)

Tabla 17. Indicadores operativos del bus en dos lecturas de la misma sesión.

Lo que no se midió. Estos valores son contadores en memoria de una sesión de demostración, no una prueba de carga: no se midió de forma sistemática la latencia por mensaje ni el número máximo de mensajes por minuto que soporta el bus. Por eso se reportan como observación y no como capacidad del sistema.

Discusión

- **Idempotencia frente a coordinación.** El reintento con HAPI apagado confirma que la idempotencia es lo que hace seguro repetir: 7 intentos, una sola entrega y ningún duplicado, sin coordinación distribuida (2PC). Esto coincide con los patrones de receptor idempotente y canal de mensajes muertos de Hohpe y Woolf (2003).
- **Disponibilidad frente a consistencia.** Con la sede MongoDB detenida, la consulta siguió respondiendo y marcó el reporte como incompleto. Es la elección AP descrita en el marco teórico: se acepta un reporte parcial y declarado a cambio de no detener la atención (Tanenbaum y Van Steen, 2017).
- **Cobertura de campos.** La cobertura muestra el límite de los datos y no del traductor: el mensaje completo llega a 56 de 57; la historia real de 1001 llega a 52 de 57 porque sus atenciones no traen todos los campos opcionales.
- **Cobertura FHIR por sede.** Los pacientes sembrados como legados solo se enlazan cuando llega un mensaje suyo; por eso se agregó la migración (`POST /hsb/migrar`), que los publica en FHIR y lleva la cobertura de 0 % a 100 % en cada sede, sin duplicar si se repite.
- **Seguridad por capas.** Las tres capas se probaron por separado: un token válido no basta para borrar datos (el rol de la base no lo permite) y una contraseña de base filtrada no sirve desde fuera (no hay puertos publicados y se exige TLS). El costo fue pequeño: el arranque genera los certificados y la verificación de un JWT RS256 no requiere consultar a la API en cada mensaje.
- **Validación FHIR.** Las 33 advertencias de `$validate` se deben a que el proyecto usa extensiones y sistemas de códigos propios que HAPI no puede resolver; no son errores de estructura. Las experiencias de Chile y Argentina usan perfiles nacionales para evitar esto (Centro Nacional en Sistemas de Información en Salud, 2022; Ministerio de Salud de la Nación Argentina, 2020).

- **Frente a las herramientas del estado del arte.** Un bus propio cubrió los requisitos con un flujo corto y probado, pero no ofrece lo que traen Mirth Connect u OpenHIM: canales gráficos, conectores ya hechos y una consola de mediación. Para una red real ese costo de construir y mantener pesaría más que en un parcial.

CONCLUSIONES

1. Se puede unificar la historia de tres sedes **sin reemplazar sus legados ni igualar sus motores**: un registro canónico de 57 campos y un repositorio con una sola interfaz bastaron para tratar PostgreSQL y MongoDB igual desde el bus y la consulta.
2. Elegir disponibilidad en la lectura (AP) mantuvo la consulta en marcha con una sede caída, a cambio de un reporte marcado como incompleto.
3. Identificadores deterministas, `PUT` en FHIR y UPSERT en las sedes hicieron idempotente la entrega; con eso, los reintentos y la recuperación tras un reinicio no necesitan transacción distribuida.
4. Separar la auditoría y las cuentas en `pg_central` permitió apagar cualquier sede sin perder la trazabilidad, y demostrar la detección de una alteración hecha directamente en la base.
5. Validar los permisos en la API, con administradores limitados a su sede y cuentas de paciente con contraseña temporal, llevó el control de acceso del prototipo más allá de los cuatro roles mínimos.
6. OAuth 2.0 con firma asimétrica permitió que el bus confíe en los tokens de la API sin compartir secretos, y junto con roles de mínimo privilegio y TLS en cada tramo protege los datos aunque una capa falle.

Limitaciones

- MongoDB corre sin *replica set*: sin transacciones multi-documento ni conmutación por fallo. Además, `guardar` lee el documento y lo reemplaza; dos escrituras simultáneas sobre el mismo paciente podrían pisarse (no hay control de concurrencia optimista).
- Una sola instancia de HAPI FHIR (persistida en PostgreSQL), sin réplica.
- Los segmentos Z (`ZPI`, `ZPV`, `ZRX`, `ZEG`) son una convención del proyecto, no un estándar nacional; los códigos REPS son sintéticos y las extensiones FHIR usan URL propias, no perfiles publicados.
- Latencia y mensajes por minuto no medidos con una prueba de carga.
- El flujo `password` de OAuth 2.0 se usa por simplicidad; OAuth 2.1 lo desaconseja y en producción se usaría *authorization code* con PKCE.
- La CA es propia: los navegadores muestran aviso hasta importarla. Los secretos van en `.env` (sin gestor de secretos) y las bases no están cifradas en reposo.

- Las cuentas semilla y los secretos de los clientes OAuth son de prueba y están documentados.
- No hay pipeline de CI/CD.

Trabajo futuro

- *Replica set* de MongoDB y control de versión del documento para escrituras concurrentes.
- Prueba de carga para medir latencia, mensajes por minuto y porcentaje de fallidos.
- Proveedor OIDC (Keycloak) con *authorization code* y PKCE, TLS mutuo entre servicios, gestor de secretos (Vault) y cifrado en reposo de las bases.
- Anclaje periódico de la cadena de auditoría en Hyperledger Fabric; Prometheus y Grafana.
- Pipeline de integración continua que corra las 60 pruebas en cada cambio.

REFERENCIAS

Bender, D., & Sartipi, K. (2013). HL7 FHIR: An agile and RESTful approach to healthcare information exchange. En *Proceedings of the 26th IEEE International Symposium on Computer-Based Medical Systems* (pp. 326–331). IEEE. https://doi.org/10.1109/CBMS.2013.6627810

Centro Nacional en Sistemas de Información en Salud. (2022). *Guía de interoperabilidad clínica con HL7 FHIR en redes asistenciales*. CENS.

HAPI FHIR Community. (2024). *HAPI FHIR: The open source FHIR API for Java* (versión 7.2.0). Smile Digital Health. https://hapifhir.io

HL7 International. (2019). *FHIR Release 4 (v4.0.1)*. https://hl7.org/fhir/R4/

Hohpe, G., & Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.

Ministerio de Salud de la Nación Argentina. (2020). *Estrategia nacional de salud digital: Guía de implementación del Perfil Argentino de FHIR*. Ministerio de Salud de la Nación.

Ministerio de Salud y Protección Social. (2021). *Resolución 866 de 2021: Por la cual se reglamenta el conjunto de elementos de datos clínicos relevantes para la interoperabilidad de la historia clínica en el país*. MinSalud.

NextGen Healthcare. (2023). *Mirth Connect integration engine technical reference guide*. NextGen Healthcare.

OpenHIE Consortium. (2022). *Open Health Information Mediator (OpenHIM) architecture specification*. Jembi Health Systems.

Hardt, D. (Ed.). (2012). *The OAuth 2.0 authorization framework* (RFC 6749). Internet Engineering Task Force. https://doi.org/10.17487/RFC6749

Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed systems* (3.ª ed.). Maarten van Steen.

ANEXOS

Anexo A. Código clave

Archivo	Qué contiene
`traductor/catalogo_57.py`	Catálogo de los 57 campos (HL7, CSV, PostgreSQL, MongoDB, FHIR)
`traductor/dominios.py`	Dominios de la Res. 866 y sinónimos por sede
`traductor/traductor.py`	Traducciones al y desde el registro canónico, derivados y cobertura
`fhir/normalizer.py`	Registro canónico → Bundle FHIR R4
`hsb/ingesta.py`, `hsb/queue_manager.py`, `hsb/main.py`	Flujo del bus, cola de reintentos, endpoints
`hsb/validator.py`	Reglas de validación antes de transformar
`db/repositorio.py`, `db/consulta_unificada.py`	Repositorios PostgreSQL y MongoDB; consulta en paralelo
`db/schema.sql`, `db/sede2/init-mongo.js`, `db/central/init.sql`	Esquemas y datos sintéticos
`app/main.py`, `app/rbac.py`, `app/audit.py`	API, permisos por rol, auditoría encadenada
`app/usuarios.py`	Cuentas en `pg_central`, PBKDF2, administradores por sede, cuenta del paciente
`app/dashboard.py`, `app/templates/`	Panel de administración, tablero y vistas
`scripts/generar_datos_prueba.py`	Generador de pacientes sintéticos enviados por el bus
`app/oauth.py`, `hsb/autorizacion.py`, `seguridad/`	Servidor OAuth 2.0, validación de tokens en el bus, TLS hacia bases y servicios
`db/seguridad/`, `scripts/generar_certificados.sh`	Roles de mínimo privilegio, `pg_hba.conf`, arranque TLS de PostgreSQL y MongoDB; CA y certificados

Tabla 18. Archivos de código clave.

Anexo B. Pruebas automatizadas

Las 60 pruebas (Tabla de pruebas por suite en Proceso de desarrollo) se ejecutan sin Docker:

```
pip install -r hsb/requirements.txt -r app/requirements.txt pytest
python -m pytest tests/ -v
...
60 passed
```

Mensaje HL7 de prueba con los 57 campos: `tests/datos/06_adt_a03_57_campos_sede1.hl7`. Guion de la demostración en vivo: `docs/guion_demostracion.md`; guía para Kali Linux: `docs/guia_kali_linux.md`.

Anexo C. Evidencias de ejecución

Extractos de las salidas de consola tomadas en vivo sobre la plataforma desplegada. Los archivos completos están en `docs/evidencias/`.

01_ejemplos_duplicado_rechazo.txt

```
# Evidencia: contrato de los ejemplos oficiales (reenvío) — 2026-11-11 00:21:56

$ curl -X POST --data-binary @ejemplos/01_adt_a01_valido_sede2.hl7 http://localhost:8001/hsb/hl7
{
    "estado": "duplicado",
    "mensaje_id": "MSG-0001",
    "estado_original": "ENTREGADO",
    "bundle_id": "bundle-pac-1007",
    "motivo": "Mensaje recibido previamente: no se vuelve a crear el paciente ni la atención."
}

$ curl -X POST --data-binary @ejemplos/03_adt_a01_invalido.hl7 http://localhost:8001/hsb/hl7
{
    "estado": "rechazado",
    "mensaje_id": "MSG-0003",
    "total_motivos": 3,
    "motivos": [
        "documento no numérico",
        "sexo no válido ('X'): debe ser M, F u O",
        "diagnóstico sin mapeo a CIE-10"
    ]
}
[...]
# Pacientes 1007 en HAPI (debe ser 1 aunque el mensaje se envió varias veces)
    "total": 1
```

02_acid.txt

```
# Evidencia: atomicidad (ACID) en PostgreSQL, sede 1 — 2026-11-11 00:21:57
$ docker exec -i pg_sede1 psql -U clinica_user -d clinica_sincelejo < db/demo_acid.sql
======================================================
PASO 1: ESTADO INICIAL (Verificar que usuario 9999 no existe)
======================================================
 documento_id | nombre_completo
--------------+-----------------
(0 rows)

 numero_visita | documento_id
---------------+--------------
(0 rows)



======================================================
PASO 2: INICIAR TRANSACCIÓN Y FORZAR ERROR
======================================================
BEGIN
INSERT 0 1
ERROR:  new row for relation "atencion" violates check constraint "atencion_clasificacion_triage_check"
DETAIL:  Failing row contains (V-TEST-FAIL, 9999, 1, null, 2026-11-11 05:21:57.453034, null, null, null,
null, null...
ROLLBACK

======================================================
PASO 3: ESTADO POSTERIOR AL ERROR (Verificar Rollback total)
======================================================
 documento_id | nombre_completo
--------------+-----------------
(0 rows)
```

```
 numero_visita | documento_id
---------------+--------------
(0 rows)


=====================================================
CONCLUSION: Atomicidad demostrada. No queda ningún registro parcial.
=====================================================
```

03_consulta_unificada_y_validacion_fhir.txt

```
# Evidencia: consulta unificada del paciente 1001 en las 3 sedes (2 motores) — 2026-11-11 00:21:57
reporte_completo: True
sedes_con_registros: [1, 2, 3]
  sede 1 postgresql disponible=True latencia=74.8 ms
  sede 2 mongodb    disponible=True latencia=83.6 ms
  sede 3 postgresql disponible=True latencia=37.9 ms
  atención V-S1-0150    postgresql 2026-11-07T11:00:00  dx=['I10:EGRESO', 'I10:INGRESO',
'R51:RELACIONADO']
  atención V-S2-0080    mongodb    2026-08-20T14:00:00  dx=['J45.9:INGRESO', 'J45.9:EGRESO']
  atención V-S3-0045    postgresql 2026-05-10T18:30:00  dx=['A09:EGRESO', 'A09:INGRESO']
cobertura 57 campos: 52 / 57 · 91.2 % · obligatorios 100.0 %


# Validación del Bundle FHIR del paciente 1001 contra HAPI ($validate)
recursos en el Bundle: 26 {'Organization': 3, 'Practitioner': 3, 'Patient': 1, 'Encounter': 3,
'Condition': 7, 'Obs...
OperationOutcome: {'warning': 33, 'information': 46}
errores: 0
```

04_health_metrics.txt

```
# Evidencia: /health y /metrics — 2026-11-11 00:22:05
[...]
$ curl http://localhost:8001/metrics
{
    "total_recibidos": 166,
    "total_entregados": 143,
    "total_rechazados": 14,
    "total_duplicados_detectados": 9,
    "total_reintentos_ejecutados": 0,
    "total_mensajes_muertos": 0,
    "mensajes_ultimo_minuto": 7,
    "latencia_media_ms": 369.87,
    "porcentaje_fallidos": 8.92,
    "estado_general": "SALUDABLE",
    "mensajes_por_estado_persistidos": {
        "RECHAZADO": 11,
        "ENTREGADO": 143
    },
    "pendientes_en_cola_reintentos": 0,
    "configuracion_reintentos": {
        "max_intentos": 7,
        "espera_base_s": 2.0
    }
}
```

05_reintentos_muerto_reencolar.txt

```
# Evidencia: reintentos con espera creciente, mensaje muerto y reencolado — 2026-11-11 00:22:06
$ docker compose stop hapi-fhir
 Container hapi_fhir_r4  Stopped


[...]
# Log del bus (docker compose logs hsb):
```

```
2026-11-11 05:22:09,729 [WARNING] Mensaje MSG-EVID-MUERTO: intento 1 falló (HAPI FHIR no disponible:
ConnectError)....
2026-11-11 05:22:11,730 [INFO] HSB reintento 2/7 del mensaje MSG-EVID-MUERTO
2026-11-11 05:22:11,853 [WARNING] Intento 2 de MSG-EVID-MUERTO falló (HAPI FHIR no disponible:
ConnectError). Sigui...
2026-11-11 05:22:15,857 [INFO] HSB reintento 3/7 del mensaje MSG-EVID-MUERTO
2026-11-11 05:22:15,972 [WARNING] Intento 3 de MSG-EVID-MUERTO falló (HAPI FHIR no disponible:
ConnectError). Sigui...
2026-11-11 05:22:23,985 [INFO] HSB reintento 4/7 del mensaje MSG-EVID-MUERTO
2026-11-11 05:22:24,150 [WARNING] Intento 4 de MSG-EVID-MUERTO falló (HAPI FHIR no disponible:
ConnectError). Sigui...
2026-11-11 05:22:40,151 [INFO] HSB reintento 5/7 del mensaje MSG-EVID-MUERTO
2026-11-11 05:22:40,329 [WARNING] Intento 5 de MSG-EVID-MUERTO falló (HAPI FHIR no disponible:
ConnectError). Sigui...
2026-11-11 05:23:12,331 [INFO] HSB reintento 6/7 del mensaje MSG-EVID-MUERTO
2026-11-11 05:23:12,465 [WARNING] Intento 6 de MSG-EVID-MUERTO falló (HAPI FHIR no disponible:
ConnectError). Sigui...
2026-11-11 05:24:16,468 [INFO] HSB reintento 7/7 del mensaje MSG-EVID-MUERTO
2026-11-11 05:24:16,656 [ERROR] Mensaje MSG-EVID-MUERTO pasa a MUERTO (cola de mensajes muertos): HAPI
FHIR no disp...
2026-11-11 05:24:18,495 [WARNING] Mensaje MSG-EVID-REINTENTO: intento 1 falló (HAPI FHIR no disponible:
ConnectErro...
2026-11-11 05:24:20,497 [INFO] HSB reintento 2/7 del mensaje MSG-EVID-REINTENTO
2026-11-11 05:24:20,601 [WARNING] Intento 2 de MSG-EVID-REINTENTO falló (HAPI FHIR no disponible:
ConnectError). Si...
2026-11-11 05:24:24,610 [INFO] HSB reintento 3/7 del mensaje MSG-EVID-REINTENTO
2026-11-11 05:24:24,798 [WARNING] Intento 3 de MSG-EVID-REINTENTO falló (HAPI FHIR no disponible:
ConnectError). Si...
2026-11-11 05:24:32,801 [INFO] HSB reintento 4/7 del mensaje MSG-EVID-REINTENTO
2026-11-11 05:24:32,927 [WARNING] Intento 4 de MSG-EVID-REINTENTO falló (HAPI FHIR no disponible:
ConnectError). Si...
2026-11-11 05:24:48,935 [INFO] HSB reintento 5/7 del mensaje MSG-EVID-REINTENTO
2026-11-11 05:24:48,978 [WARNING] Intento 5 de MSG-EVID-REINTENTO falló (HAPI FHIR no disponible:
ConnectError). Si...
2026-11-11 05:25:20,987 [INFO] HSB reintento 6/7 del mensaje MSG-EVID-REINTENTO
2026-11-11 05:25:21,186 [WARNING] Intento 6 de MSG-EVID-REINTENTO falló (HAPI FHIR no disponible:
ConnectError). Si...
2026-11-11 05:26:25,187 [INFO] HSB reintento 7/7 del mensaje MSG-EVID-REINTENTO
2026-11-11 05:26:30,371 [INFO] Mensaje MSG-EVID-REINTENTO ENTREGADO tras 7 intentos (132005 ms)
[...]
[2026-11-11 00:26:31] Reencolar el mensaje muerto con HAPI ya encendido:
$ curl -X POST http://localhost:8001/hsb/reencolar/MSG-EVID-MUERTO
{
    "mensaje": "Mensaje MSG-EVID-MUERTO reencolado",
    "nuevo_estado": "REINTENTANDO"
}
  MSG-EVID-MUERTO ENTREGADO intentos= 1
2026-11-11 05:26:34,214 [INFO] Mensaje MSG-EVID-MUERTO ENTREGADO tras 1 intentos (2607 ms)
```

06_pruebas_automatizadas.txt

```
# Evidencia: pruebas automatizadas — 2026-11-11 12:23
$ python -m pytest tests -v
[... 60 pruebas, todas PASSED ...]
============================ 60 passed in 8.44s ==============================
```

08_seguridad_oauth_bd_tls.txt

```
# Evidencia de seguridad v3 · 2026-11-11 12:21 · VM Kali Linux (Docker)
# 1. OAuth 2.0: token de servicio (client_credentials) y su contenido
  Token OAuth 2.0 listo ($TOKEN)
    algoritmo: RS256 · kid: a11f39b8f2246e1b
    cliente: herramientas_qa · audiencia: ['hsb'] · scope: hsb.ingesta hsb.admin
    vence: 12:36:44
# 2. OAuth 2.0: el bus exige token válido
$ $CURL -o /dev/null -w "sin token: %{http_code}\n" -X POST --data-binary
@ejemplos/01_adt_a01_valido_sede2.hl7 htt...
sin token: 401
```

```
$ $CURL -o /dev/null -w "con token: %{http_code}\n" -H "Authorization: Bearer $TOKEN" -X POST --data-
binary @ejempl...
con token: 200
$ $CURL -o /dev/null -w "token de usuario (aud api): %{http_code}\n" -H "Authorization: Bearer $U" -X
POST --data-b...
token de usuario (aud api): 401
$ $CURL -H "Authorization: Bearer $S1" -X POST --data-binary @ejemplos/04_sede3_legado.csv
"https://localhost:8001/...
{"detail":{"error":"insufficient_scope","error_description":"El cliente sistema_sede1 pertenece a la
sede 1 y no pu...
$ $CURL -u herramientas_qa:falso -d grant_type=client_credentials https://localhost:8000/oauth/token
{"error":"invalid_client","error_description":"Cliente desconocido o secreto incorrecto"}
# 3. Bases de datos: mínimo privilegio, auditoría de solo agregar y TLS obligatorio
== 1. Las bases no publican puertos (solo API, bus y HAPI en 127.0.0.1)
api_gateway    0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
hapi_fhir_r4    127.0.0.1:8090->8080/tcp
hsb_bus    0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp
mongo_sede2
pg_central
pg_sede1
pg_sede3
== 2. Mínimo privilegio · el bus (hsb_escritor) intenta BORRAR un paciente de la sede 1
ERROR:  permission denied for table usuario
== 3. Mínimo privilegio · la API (api_lector) intenta MODIFICAR la sede 3 (y luego solo lee)
ERROR:  permission denied for table usuario
lectura permitida: 53 pacientes
== 4. Auditoría de solo agregar · la API (api_central) intenta alterar y borrar la cadena
ERROR:  permission denied for table evento_auditoria
ERROR:  permission denied for table evento_auditoria
== 5. MongoDB (sede 2) · api_lector intenta insertar y hsb_escritor intenta borrar
MongoServerError: not authorized on clinica_cartagena to execute command
MongoServerError: not authorized on clinica_cartagena to execute command
== 6. Cifrado obligatorio · conexión SIN TLS a PostgreSQL y a MongoDB
FATAL:  pg_hba.conf rejects connection for host "::1", user "hsb_escritor", database
"clinica_sincelejo", no encryp...
MongoServerSelectionError: connection <monitor> to 127.0.0.1:27017 closed
== 7. Quién se conecta a cada base y con qué cifrado (registro de PostgreSQL)
   pg_sede1: user=hsb_escritor database=clinica_sincelejo SSL enabled (protocol=TLSv1.3,
cipher=TLS_AES_256_GCM_SHA...
   pg_sede3: user=hsb_escritor database=clinica_corozal SSL enabled (protocol=TLSv1.3,
cipher=TLS_AES_256_GCM_SHA38...
   pg_central: user=api_central database=clinica_central SSL enabled (protocol=TLSv1.3,
cipher=TLS_AES_256_GCM_SHA3...
   pg_central: user=hapi_srv database=hapi SSL enabled (protocol=TLSv1.3, cipher=TLS_AES_256_GCM_SHA384,
bits=256)
# 4. Cifrado entre servidores
$ openssl s_client -connect localhost:8001 -servername hsb -CAfile certs/ca.pem </dev/null 2>/dev/null |
grep -E "s...
subject=C=CO, O=Red IPS Sucre-Bolivar, CN=hsb
issuer=C=CO, O=Red IPS Sucre-Bolivar, OU=Plataforma de Interoperabilidad Clinica, CN=CA Plataforma
Interoperabilidad
Protocol: TLSv1.3
Verify return code: 0 (ok)
$ $CURL -o /dev/null -w "HAPI por HTTPS: %{http_code}\n" https://localhost:8090/fhir/metadata
HAPI por HTTPS: 200
$ curl -s -o /dev/null -w "HAPI por HTTP plano: %{http_code}\n" http://localhost:8090/fhir/metadata
HAPI por HTTP plano: 400
```

Anexo D. Usuarios de prueba

Todos los datos son sintéticos. Las contraseñas son de prueba y están documentadas en el README.

Usuario	Contraseña	Rol	Qué puede hacer
paciente_carlos	Paciente123!	paciente	Su historia (documento 1001), su Bundle FHIR y quién accedió a su historia

Usuario	Contraseña	Rol	Qué puede hacer
admision_laura	Admision123!	admisionista	Buscar en qué sedes existe un paciente, admitir con triage, ver envíos al bus. No ve diagnósticos
medico_mateo	Medico123!	médico	Historia unificada; diagnóstico, fórmula y egreso con sello
auditor_elena	Auditor123!	auditor	Reporte de HC, estado de la plataforma, verificador; panel en modo consulta
admin_plataforma	Admin123!	administrador	Panel de administración y usuarios de toda la red
admin_sincelejo, admin_cartagena, admin_corozal	Admin123!	administrador de sede	Panel; crea y gestiona médicos, admisionistas y pacientes de su sede
Documento del paciente admitido	Temporal (se muestra una vez)	paciente	Portal del paciente; debe cambiar la contraseña en el primer ingreso

Tabla 19. Usuarios de prueba.

Anexo E. Cómo ejecutar

```
bash desplegar.sh --desde-cero --con-datos --verificar   # certificados, roles, servicios y 70/70

# Vistas: https://localhost:8000   HSB: https://localhost:8001   HAPI (solo local):
https://localhost:8090/fhir
source scripts/sesion_demo.sh    # deja $CURL (HTTPS con la CA) y $TOKEN (OAuth 2.0)

# Ejemplos oficiales (el bus exige token)
$CURL -H "Authorization: Bearer $TOKEN" -X POST --data-binary
@ejemplos/01_adt_a01_valido_sede2.hl7 \
      https://localhost:8001/hsb/hl7
$CURL -H "Authorization: Bearer $TOKEN" -X POST --data-binary @ejemplos/04_sede3_legado.csv \
      "https://localhost:8001/hsb/csv?sede=3"

# Seguridad de las bases (mínimo privilegio, auditoría de solo agregar, TLS obligatorio)
bash scripts/demo_seguridad_bd.sh

# Reintento: HAPI caído
docker compose stop hapi-fhir   # enviar un mensaje, luego:
docker compose start hapi-fhir  # el bus lo entrega solo

# Caída de una sede
docker stop mongo_sede2         # luego: docker start mongo_sede2

# Alteración de la auditoría
docker exec -it pg_central psql -U clinica_user -d clinica_central \
  -c "UPDATE evento_auditoria SET usuario_sistema='intruso' WHERE indice = 3"

# ACID en PostgreSQL
docker exec -i pg_sede1 psql -U clinica_user -d clinica_sincelejo < db/demo_acid.sql
```

Anexo F. Uso declarado de inteligencia artificial

Etapa	Herramienta	Uso	Revisión humana
Fases 1 a 5 (v1)	Antigravity (Google DeepMind)	Código Mermaid de los diagramas, estructura del repositorio, esquemas SQL, normalizador, parsers, validador, plantillas, pruebas y borradores de los informes	Revisión de coherencia de los diagramas, verificación de los 4 ejemplos oficiales, pruebas de 403, alteración y caída de sede

Etapa	Herramienta	Uso	Revisión humana
Mejora v2	Claude Code (Anthropic)	Traductor de 57 campos, adaptador MongoDB, cola de reintentos, auditoría persistente, panel de administración, gestión de usuarios, vistas, generador de datos y pruebas	Pruebas en vivo en Kali Linux con Docker (ejemplos, duplicado, reintento, caída de sede, alteración, `$validate`) y 44 pruebas automatizadas
Mejora v3 (seguridad)	Claude Code (Anthropic)	Servidor OAuth 2.0, autorización en el bus, roles de las bases, TLS y certificados, pasarela FHIR y pruebas	Despliegue sobre datos existentes en Kali Linux; pruebas de permisos, TLS y tokens (Anexo C); 60 pruebas y 70/70 comprobaciones
Imagen de portada del inicio de sesión	Asistente ATLAS	Generación de la imagen	Selección por los integrantes

Tabla 20. Uso declarado de IA (detalle en `docs/bitacora.md`).

Código abierto usado: FastAPI, Uvicorn, HTTPX, Jinja2, PostgreSQL, MongoDB y PyMongo, HAPI FHIR (Apache 2.0), Docker Compose. Cada integrante debe poder explicar y modificar en vivo lo que entregó.

AVANCE INICIAL

Estado de los entregables de la Fase 1 (análisis, UML y diseño), entregada el 8 de noviembre de 2026, y su actualización en la versión 2:

- **Repositorio y estructura base:** repositorio Git con commits de ambos integrantes desde la Fase 1, estructura de carpetas (`app/`, `hsb/`, `fhir/`, `db/`, `tests/`, `ejemplos/`, `docs/`, `infra/`), `README.md`, `.env.example` y `docker-compose.yml`. Completo.
- **Cuatro diagramas UML en Mermaid** (casos de uso, secuencia, clases y estados del mensaje) con tablas de coherencia entre ellos. Completo; actualizados a la v2 (MongoDB, traductor, cuentas).
- **Arquitectura y justificación de componentes:** diagrama de arquitectura y tabla componente / requisito / alternativas / razón. Completo; actualizado a siete contenedores.
- **Mapeo de campos:** tabla de la Fase 1 con 24 filas representativas (mínimo exigido: 15) y, en la v2, el mapeo completo de los 57 campos generado desde el catálogo (`docs/mapeo_57_campos.md`).
- **Entorno Docker:** esqueleto de `docker-compose.yml` en la Fase 1; en la v2, siete contenedores que arrancan con un solo comando.
- **Marco teórico y pregunta de autoevaluación** de la Fase 1 (sede 2 caída: el médico ve la historia parcial con aviso). Completo.
- **Pendiente de la Fase 1:** la evidencia de ejecución del repositorio de referencia del docente quedó marcada como pendiente en el informe de esa fase.

VIABILIDAD TÉCNICA: ESCALABILIDAD Y CUMPLIMIENTO NORMATIVO

Esta sección distingue lo que el prototipo implementa y se probó de lo que se propone para una operación real en la red de IPS.

Escalabilidad

Se marca con **[Implementado]** lo que existe y se probó, y con **[Propuesto]** lo que no se implementó.

- **[Implementado]** Siete contenedores con Docker Compose; cada uno puede caer sin detener a los demás (se probó con `mongo_sede2` y `hapi-fhir`).
- **[Implementado]** Consulta unificada en paralelo: una sede caída cuesta su tiempo de espera (3 s), no se suma al de las otras.
- **[Implementado]** Cola de reintentos en segundo plano con estado persistido en cada sede.
- **[Implementado, sin evidencia de ejecución]** Manifiestos de Kubernetes para Minikube en `infra/k8s/`.
- **[Propuesto]** Réplicas del HSB detrás de un balanceador y un broker (Kafka o RabbitMQ) particionado por sede; autoescalado por tamaño de cola.
- **[Propuesto]** Réplicas de lectura en PostgreSQL; *replica set* en MongoDB; Citus si una sede crece a millones de atenciones; HAPI en más de una instancia.
- **[Propuesto]** Prometheus y Grafana sobre los `/metrics` existentes; prueba de carga.

Aspecto	Implementado	Propuesto (no implementado)
Ingesta	Un HSB con cola en hilo de segundo plano y estado en las bases	Réplicas del HSB y broker particionado por sede
Datos	2 PostgreSQL y 1 MongoDB, cada uno un nodo único	Réplicas de lectura; *replica set*; Citus
FHIR	Una instancia de HAPI persistida en PostgreSQL, solo en red local	Varias instancias de HAPI
Seguridad	OAuth 2.0 RS256 (API y bus), roles de mínimo privilegio, TLS 1.3 en cada tramo, CA propia	Keycloak/OIDC con PKCE, TLS mutuo, gestor de secretos, cifrado en reposo
Auditoría	Cadena SHA-256 en `pg_central`	Anclaje en Hyperledger Fabric (bonus no hecho)
Despliegue	Docker Compose; manifiestos para Minikube	Autoescalado del HSB por tamaño de cola

Tabla 21. Viabilidad técnica: implementado frente a propuesto.

Qué escalar primero ante 100 veces más mensajes: el HSB. Hoy hace el primer intento de entrega de forma síncrona y tiene un solo hilo de reintentos por proceso. Lo primero sería poner un broker delante para responder `202` de inmediato y repartir el trabajo entre varias réplicas del HSB; luego HAPI (más instancias sobre PostgreSQL) y por último las bases de cada sede.

Cumplimiento normativo

- **[Implementado]** Solo datos sintéticos, también los códigos REPS de prestador.
- **[Implementado]** Ley 1581 de 2012: el paciente ve quién accedió a su historia, con qué rol y cuándo (acceso y finalidad); el admisionista no ve datos clínicos (mínimo privilegio); la auditoría encadenada impide reescribir sin dejar rastro.
- **[Implementado]** Resolución 866 de 2021: los 57 campos están en el catálogo y se mide su cobertura por paciente con la lista de faltantes.

- **[Implementado]** Contraseñas PBKDF2-SHA256 con sal por usuario; contraseña temporal con cambio obligatorio; cada alta, desactivación o restablecimiento queda auditado.
- **[Implementado]** OAuth 2.0: tokens RS256 con audiencia y scopes; el HSB exige token y un sistema de sede solo escribe en su sede.
- **[Implementado]** Mínimo privilegio en las bases (un rol por servicio; la auditoría es de solo agregar) y cifrado en tránsito con TLS 1.3 entre todos los servidores; las bases no tienen puertos publicados.
- **[Propuesto]** Cifrado de las bases en reposo, gestor de secretos y proveedor de identidad externo (Keycloak/OIDC).
- **[Propuesto]** Registro del consentimiento del paciente (recurso FHIR Consent) y políticas de retención de la historia clínica.

INTEROPERABILIDAD BASADA EN DATOS: ENFOQUE SEMÁNTICO, TÉCNICO Y ORGANIZACIONAL

La interoperabilidad no se limita a mover mensajes: los datos deben llegar con el mismo significado, completos y trazables. Esta sección describe cómo la plataforma trata cada dimensión. Los pilares técnico y semántico se demostraron; el organizacional solo se menciona, porque el gobierno de datos se evalúa en el curso de Arquitectura TI.

1. Calidad y Gestión de Datos

- **Modelo canónico de 57 campos** (`traductor/catalogo_57.py`): un único catálogo define origen, destino, obligatoriedad y dominio de cada campo; las tres sedes se traducen contra él.
- **Validación antes de transformar** (`hsb/validator.py`): documento, nombre, sexo, sede, fechas, CIE-10 y LOINC; un mensaje inválido se rechaza con todos sus motivos y no llega a FHIR.
- **Cobertura medida:** por paciente (campos con valor sobre 57 y obligatorios sobre 23, con la lista de faltantes) y por sede (pacientes con recurso en HAPI).
- **Unicidad e idempotencia:** identificadores deterministas en FHIR y llaves de UPSERT en las sedes; la búsqueda de admisión detecta datos distintos del mismo documento entre sedes (posible duplicado).
- **Trazabilidad:** estado de cada mensaje en `mensaje_bus` de su sede y eventos de acceso en la cadena de auditoría.

2. Interoperabilidad Semántica

- **CIE-10** en `Condition.code` para los diagnósticos 33 a 39 y **LOINC** en `Observation.code` para laboratorio.

- **Tabla de terminología** `ejemplos/05_ref_terminologia.csv`: convierte los textos locales del CSV de la sede 3 a CIE-10; lo que no se puede mapear se rechaza con motivo.
- **Dominios con sinónimos** (`traductor/dominios.py`): tipo de documento, entorno, vía de ingreso, triage, condición de salida, entre otros, de modo que `E` (HL7), `EMER` o `01` significan lo mismo.
- **Extensiones FHIR propias** para los campos de la Res. 866 sin lugar en el recurso base (no se usó un perfil nacional publicado).
- **No implementado:** servidor de terminología, ConceptMap y ValueSet de FHIR; se proponen como trabajo futuro.

Ejemplo de un campo de punta a punta, el **14, municipio de residencia (código DANE)**:

Paso	Representación
HL7 v2	`PID-11.9` (noveno componente de la dirección), por ejemplo `70001`
Registro canónico	`usuario.municipio_residencia = "70001"`
PostgreSQL (sedes 1 y 3)	`usuario.municipio_residencia VARCHAR(5)`
MongoDB (sede 2)	`pacientes.residencia.municipio_dane`
FHIR R4	`Patient.address[0].district`

Tabla 22. Recorrido del campo 14 por los cinco formatos.

3. Interoperabilidad Técnica

- **Entradas:** HL7 v2 en texto por HTTP POST (ADT^A01, ADT^A03, ORU^R01 y RDE^O11 en sus segmentos de medicamento), CSV legado con `;` y JSON de las vistas.
- **Segmentos Z** `ZPI`, `ZPV`, `ZRX` y `ZEG` para los datos de la Res. 866 que HL7 v2.5 no trae.
- **Salida estándar:** FHIR R4 por REST en JSON, con Bundle `transaction` y `PUT` de identificadores deterministas; validación con `$validate` de HAPI.
- **Dos motores de base** con modelos distintos (relacional y documental) integrados mediante el registro canónico y el repositorio con interfaz única.
- **Desacople:** las vistas y los legados no comparten base; todo pasa por el HSB y la API. Cada servicio expone `/health` y `/metrics`.
- **No implementado:** suscripciones FHIR, perfiles IHE y mensajería asíncrona con broker.

4. Gobernanza Organizacional

Solo se menciona, como pide la guía. En una red real harían falta acuerdos entre las IPS sobre:

- La definición y el versionado de los segmentos Z, los códigos locales y la tabla de terminología.
- Los roles y sus permisos: qué ve cada perfil (en el prototipo, paciente, admisionista, médico, auditor y administrador general o de sede).
- La revisión periódica de la auditoría y de los accesos a la historia clínica.
- El cumplimiento de la Ley 1581 de 2012 (autorización, finalidad, acceso) y la custodia de los datos.

5. Herramientas de Integración Técnica

- **HAPI FHIR R4 7.2.0** (código abierto, Java): **usado** como servidor FHIR; recibe los Bundles `transaction` y valida con `$validate`.
- **HSB propio en FastAPI** (Python): **usado** como bus; traducción, validación, cola, reintentos e idempotencia.
- **Mirth Connect** (NextGen Healthcare, 2023): **evaluado, no usado**; motor HL7 con canales gráficos. Se descartó por agregar otra plataforma Java y una configuración que el equipo no podía modificar en vivo con la misma soltura.
- **OpenHIM** (OpenHIE Consortium, 2022): **evaluado, no usado**; su idea de separar la mediación (seguridad, auditoría) del almacenamiento sí se aplicó con `pg_central`.
- **Apache Camel**: **evaluado, no usado**; marco de integración en Java con componentes HL7 y FHIR.
- **Docker Compose**: **usado** para el despliegue de un comando.

6. Herramientas de Integración Semántica

- **Catálogo y dominios propios** (`catalogo_57.py`, `dominios.py`): **usados** para normalizar los valores de la Res. 866.
- **Tabla de terminología del docente** (`05_ref_terminologia.csv`): **usada** para mapear los textos locales a CIE-10.
- **Validador con expresiones regulares** para el formato de CIE-10 y LOINC: **usado**. Verifica el formato del código, no su existencia en el catálogo oficial.
- **Servidores de terminología** (por ejemplo, el módulo de terminología de HAPI con ValueSet y ConceptMap): **no usados**; se proponen para validar que cada código exista.

7. Compatibilidad de Herramientas con Estándares de Interoperabilidad

La matriz resume la compatibilidad de herramientas de integración con los estándares HL7 v2.x, HL7 CDA, HL7 FHIR y perfiles IHE. Las filas de herramientas de terceros provienen de la comparación de referencia de la plantilla del curso y no fueron verificadas por el equipo; la última fila describe la plataforma de este proyecto. Símbolos: ✓ soporte nativo, ◑ parcial, ✗ no soportado.

Herramienta	HL7 v2.x	HL7 CDA	HL7 FHIR	IHE perfiles
HAPI FHIR Server	✗	✗	✓	◑
Smile CDR	✓	✓	✓	✓
Mirth Connect	✓	✓	✓	◑
OpenHIM	✓	◑	✓	✓
Apache Camel + camel-fhir	✓	◑	✓	◑
Kong API Gateway	✗	✗	✓	✗
Redox (SaaS)	✓	✓	◑	✗
Rhapsody Integrator	✓	✓	✓	✓
GCP Healthcare API / Azure HDS	✓	◑	✓	✗
Snowstorm	✗	✗	✓	✗

Herramienta	HL7 v2.x	HL7 CDA	HL7 FHIR	IHE perfiles
Ontoserver	✕	✕	✓	✕
Apelon DTS	✕	✕	◑	✕
Plataforma de este proyecto (HSB propio)	✓ ADT^A01, ADT^A03, ORU^R01; RDE^O11 parcial	✕	✓ R4, Bundle transaction	✕

Tabla 23. Compatibilidad de herramientas de integración con los estándares de interoperabilidad en salud.

Observaciones. La plataforma cubre los dos estándares que exige el parcial: HL7 v2 como entrada de los legados y FHIR R4 como salida. No trabaja con documentos CDA ni con perfiles IHE (por ejemplo, PIX/PDQ para identidad de pacientes o ATNA para auditoría); la identidad se resuelve con el número de documento y la auditoría con la cadena propia. Adoptar IHE sería el paso natural para conectarse con una red regional.

VALIDACIÓN DE MODELOS UML

Métodos

Los modelos UML se validaron en tres niveles, sin herramientas de modelado comerciales:

- **Coherencia entre diagramas:** cada archivo de `docs/fase1/diagramas/` tiene una tabla que cruza actores, participantes, clases y estados (por ejemplo, cada caso de uso con sus participantes en la secuencia, sus clases y los estados que produce). La revisión fue cruzada entre los dos integrantes.
- **Renderizado:** los cuatro diagramas y la arquitectura se escriben en Mermaid y se renderizan a PNG; un error de sintaxis impide el renderizado.
- **Comprobación contra el código:** pruebas automatizadas que comparan el modelo con la implementación, la comparación de los estados del diagrama con los de `hsb/queue_manager.py` y la validación FHIR de los Bundles generados.

Elemento	Regla de validación	Herramienta
Casos de uso	Cada caso de uso tiene un endpoint o función que lo implementa y un participante en la secuencia	Tabla de coherencia en `01_casos_uso.md`; revisión cruzada
Secuencia	Cada participante es una clase o un componente desplegado; cada mensaje a `mensaje_bus` produce un estado del diagrama 4	Tabla de coherencia en `02_secuencia_hsb.md`
Clases	Cada atributo del dominio corresponde a una columna de `db/schema.sql` y a un campo del catálogo de 57	`pytest`: `test_schema_sql_tiene_todas_las_columnas_del_catalogo`
Clases (traductor)	Hay 57 campos numerados y cada uno tiene destino PostgreSQL, MongoDB y FHIR	`pytest`: `test_hay_57_campos_numerados_y_cada_uno_tiene_destino`
Estados	Los 7 estados del diagrama son los valores de `MensajeBus.estado` y los que escribe `hsb/queue_manager.py`	Revisión del código; pruebas de reintento, muerto y reencolar
Recursos FHIR	El Bundle generado cumple la especificación FHIR R4	Operación `$validate` de HAPI FHIR
Sintaxis	Los diagramas compilan	Renderizado Mermaid a PNG

Tabla 24. Reglas y herramientas usadas para validar los modelos UML.

Resultados

El Bundle unificado del paciente 1001 (26 recursos) validó en HAPI con 0 errores, 33 advertencias y 46 mensajes informativos; las advertencias se deben a las extensiones y sistemas de códigos propios. Las pruebas que comparan el esquema SQL y el catálogo con el modelo pasan (forman parte de las 44). Los siete estados del diagrama coinciden con los que escribe la cola del bus.

La revisión entre fases encontró inconsistencias entre el modelo y el código, que se corrigieron y se anotaron en la bitácora: el modelo de datos cubría unos 15 de los 57 campos y suponía el mismo motor en todas las sedes (se pasó al esquema de 57 campos y a MongoDB en la sede 2); la auditoría del diagrama era persistente pero el código la guardaba en memoria (se pasó a `pg_central`); el estado REINTENTANDO terminaba en MUERTO antes de que HAPI volviera (se cambió la política de reintentos); y la reentrega de un mensaje duplicaba diagnósticos, en contra del principio de idempotencia del diagrama de secuencia (identificadores deterministas y UPSERT). Los diagramas se actualizaron a la v2.

Discusión

Validar el modelo contra el código con pruebas automáticas fue más útil que una validación solo visual: la prueba que compara `db/schema.sql` con el catálogo falla en cuanto un campo del modelo no tiene columna, y así el diagrama de clases no puede quedar desactualizado sin que se note. La limitación es que la coherencia entre diagramas se revisa a mano con las tablas, sin una herramienta de validación formal (por ejemplo, reglas OCL), y que el validador FHIR revisa la estructura pero no la existencia de los códigos CIE-10 y LOINC en sus catálogos oficiales.